

ModNet: A Self-Growing Modular Recurrent Architecture for Layer-Free Computation

An evolutionary framework that discovers structure from first principles

Masoud Azizi¹

Department of Computer Science
Faculty of Mathematical Sciences
University of Tabriz
Tabriz, Iran

September 17, 2026

¹Corresponding author. Email: mablue92@gmail.com

Abstract

We present MODNET, a self-growing modular recurrent network whose topology is discovered through evolution rather than imposed by a designer. The architecture has no predefined layers: each node computes a weighted sum of its inputs followed by a threshold activation, and any node may connect to any other node, including itself. Starting from a minimal random topology, the network grows, prunes, and reorganizes itself through a compact evolutionary loop to solve tasks ranging from Boolean logic (XOR, parity of four and six bits) to continuous regression (sinusoids, quadratic surfaces). We report exact solutions (100% accuracy) on XOR and 4-bit parity, and root-mean-square errors below 0.05 on continuous targets, using fewer than 30 nodes per network. Across all tasks we observe that cross-module connectivity, recurrence, and sparse wiring emerge spontaneously as the task demands, without any explicit regularization or architectural prior. Our results suggest that layer-free computation driven by a minimal evolutionary process can rediscover structural principles typically imposed by human designers.

Keywords: neuroevolution, self-growing networks, layer-free architectures, modular recurrent networks, Boolean function learning, sparse connectivity.

Contents

1	Introduction	7
1.1	Motivation	7
1.2	Background and Historical Context	7
1.2.1	The XOR Problem and the Limits of Linear Models	7
1.2.2	Evolutionary Approaches to Network Design	8
1.2.3	Modularity in Neural Networks	8
1.2.4	Sparse Connectivity and Pruning	8
1.2.5	Self-Organizing and Developmental Systems	9
1.2.6	Universality of Neural Computation	9
1.3	The Present Work	9
1.4	Contributions	10
1.5	Organization of the Thesis	10
2	Related Work	11
2.1	Neuroevolution: Evolving Neural Network Topologies	11
2.2	Modular Neural Networks	12
2.3	Self-Organizing and Developmental Systems	12
2.4	Biological Inspiration: Criticality and Small-World Topology	13
2.5	Boolean Function Learning and Parity	14
2.6	Sparse and Efficient Neural Computation	14
2.7	Neural Architecture Search	15
2.8	Universal Computation and Neural Cellular Automata	15
2.9	Summary and Positioning	16
3	Method	17
3.1	Design Principles	17
3.2	Node Model	18
3.3	Modular Structure	18
3.3.1	Module Sizes	19
3.3.2	Output Module	19
3.3.3	Measuring Modularity	19
3.4	Layer-Free Connectivity	19
3.5	Recurrence	20
3.6	Fitness Evaluation	20
3.7	Evolutionary Algorithm	21
3.7.1	Mutation	21
3.7.2	Crossover	22
3.7.3	Growth (Birth)	22
3.7.4	Pruning	22

3.7.5	Selection	23
3.8	Complexity Analysis	23
3.9	Summary of Hyperparameters	23
3.10	Reproducibility	24
4	Experimental Setup	25
4.1	Benchmark Tasks	25
4.1.1	Boolean Function Learning	25
4.1.2	Continuous Regression	25
4.2	Data Encoding	27
4.2.1	Boolean Inputs	27
4.2.2	Continuous Inputs	27
4.2.3	Outputs	27
4.3	Evaluation Procedure	28
4.4	Hyperparameter Settings	28
4.5	Reproducibility	28
4.6	Computational Cost	29
4.7	Summary of Reproducibility Artifacts	29
5	Results	31
5.1	Overview	31
5.2	Boolean Function Learning	31
5.2.1	XOR (2-bit)	31
5.2.2	Parity (4-bit)	32
5.2.3	Parity (6-bit)	33
5.3	Continuous Regression	36
5.3.1	$\sin(2\pi x)$	36
5.3.2	$\sin(4\pi x)$	36
5.3.3	Circle ($x^2 + y^2$)	37
5.4	Emergent Structural Properties	38
5.4.1	Cross-Module Integration	38
5.4.2	Recurrence	39
5.4.3	Sparsity	39
5.4.4	Growth Regimes	40
5.5	Failure Analysis: Parity-6	40
5.5.1	What Happened	40
5.5.2	Why It Failed	40
5.5.3	Implications	41
5.6	Summary of Key Findings	41
6	Discussion	43
6.1	Interpretation of the Main Findings	43
6.1.1	Layer-Free Computation Is Feasible	43
6.1.2	Emergent Structure Is Not Imposed	43
6.1.3	Growth Reflects Task Complexity	44
6.2	Relationship to Related Work	44
6.2.1	Comparison with NEAT and HyperNEAT	44
6.2.2	Comparison with Self-Organizing Systems	45
6.2.3	Comparison with Neural Cellular Automata	45

6.3	Limitations	45
6.3.1	Limited Scale	45
6.3.2	Limited Task Diversity	45
6.3.3	No Theoretical Guarantees	46
6.3.4	Dependence on Hyperparameters	46
6.3.5	Determinism and Reproducibility	46
6.4	Threats to Validity	46
6.4.1	Internal Validity	46
6.4.2	External Validity	47
6.4.3	Construct Validity	47
6.5	Future Work	47
6.5.1	Scaling to Larger Networks	47
6.5.2	Combining Evolution with Gradient Descent	48
6.5.3	Extending to Structured Inputs	48
6.5.4	Theoretical Analysis	48
6.6	Broader Implications	49
6.7	Summary	49

List of Figures

3.1	Node model. Each node computes a weighted sum of its sources plus a bias, then applies a Heaviside step function. The self-loop (dashed arrow) indicates that the node may receive input from itself, which introduces recurrence.	18
3.2	Modular layer-free architecture. Nodes (circles) are grouped into modules (colors), but cross-module connections (bold arrows) are unrestricted. The dashed self-loop on a node in Module 1 illustrates recurrence. Cross-module edges may connect any pair of modules, including from the output module back to earlier modules.	19
3.3	Pruning removes nodes whose activity is either below α_{lo} or above α_{hi} across all training samples (dashed gray nodes). The remaining nodes and connections are re-indexed to form a compact network.	23
4.1	Illustration of the two regression task families. Left: the sinusoid used in <code>sin2pi</code> and <code>sin4pi</code> . Right: a slice of the quadratic surface used in <code>circle</code>	27
5.1	The final network evolved for the XOR task. Nodes are colored by module (see Chapter 3 for the color legend). Cross-module connections dominate: 14 of the 17 active connections cross module boundaries. The network uses only 27% of the available connections (17 of 64), and contains 2 recurrent self-loops.	32
5.2	Learning curve for XOR. The fitness improves from -0.25 (chance level) to 0 (perfect accuracy) in 32 generations. The rapid jump indicates that once a topology capable of computing XOR is found, it is immediately perfected.	32
5.3	The final network evolved for 4-bit parity. The output module (right) contains a single node, which receives input from all three earlier modules. Recurrence is present in 9 of the 12 nodes, indicating that the network uses temporal computation extensively.	33
5.4	Learning curve for 4-bit parity. The fitness improves monotonically from -0.50 (chance level for a balanced Boolean function) to 0 over 320 generations. The progression is smooth, with visible plateaus at each intermediate fitness level (corresponding to solving 8, 10, 12, and 14 of the 16 cases).	34
5.5	The best network evolved for 6-bit parity. It reached 93.8% accuracy but did not reach the exact-solution threshold. The network is smaller (10 nodes) than the successful Parity-4 network (12 nodes), suggesting that it may be under-parameterized for this task.	34
5.6	Learning curve for 6-bit parity. The fitness improves from -0.50 to -0.0625 over 1500 generations but does not reach zero. Unlike Parity-4, which reached 100% accuracy in 320 generations, Parity-6 shows no sign of converging to an exact solution.	35

5.7	The final network evolved for $\sin(2\pi x)$. This is the largest network in our experiments (30 nodes), and it uses 74% of its active connections for cross-module communication. The 13 recurrent self-loops indicate that the network uses temporal computation to reproduce the periodic output.	36
5.8	Predictions of the evolved network on the 32 training samples for $\sin(2\pi x)$. RMSE = 0.0412.	36
5.9	The final network evolved for $\sin(4\pi x)$. This network is substantially smaller than the <code>sin2pi</code> network (19 vs. 30 nodes), suggesting that the search process terminated before reaching the same level of structural complexity.	37
5.10	Learning curve for $\sin(4\pi x)$. The network was still improving at generation 1500, suggesting that with additional generations it might have reached the success threshold.	37
5.11	The final network evolved for $x_1^2 + x_2^2$. Despite having the largest input dimension (8 bits, encoding two continuous variables), this task was solved in fewer generations than <code>sin2pi</code> , suggesting that the structure of the target function matters more than the input dimensionality.	38
5.12	Predictions of the evolved network on a slice of the $x_1^2 + x_2^2$ surface at fixed $x_2 = 0.5$. RMSE = 0.0222.	38
5.13	Proportion of cross-module connections among active connections, for each task. The horizontal line at 0.75 corresponds to the theoretical baseline for a random network with $M = 4$ equal-sized modules.	39

List of Tables

2.1	Positioning of MODNET relative to related architectures.	16
3.1	Hyperparameters of MODNET.	24
4.1	Summary of benchmark tasks.	26
4.2	Task-specific hyperparameters.	28
4.3	Wall-clock time per task.	29
4.4	Reproducibility artifacts per task.	30
5.1	Summary of experimental results. All tasks use the same architecture and evolutionary algorithm; only the number of time steps T and the encoding differ. . . .	31
5.2	Input-output mapping for the evolved XOR network.	33
5.3	Complete input-output mapping for the evolved Parity-4 network. All 16 input combinations are classified correctly.	35
5.4	Misclassified cases for the Parity-6 network. All four errors occur for inputs with weight ≥ 4 , and three of the four have weight exactly 5.	35
5.5	Recurrence count by task. The second column shows the number of nodes with self-loops; the third column shows the fraction of nodes that are recurrent.	39
5.6	Density of evolved networks. Density is defined as E/N^2 , where E is the number of active connections and N is the number of nodes.	40
5.7	Follow-up experiments on Parity-6. Neither configuration reached 100% accuracy within the budget.	41

Chapter 1

Introduction

1.1 Motivation

The design of artificial neural networks has historically been guided by human intuition. From the earliest perceptron models of Rosenblatt [52] to modern deep architectures [32], researchers have manually specified the number of layers, the connectivity patterns, the activation functions, and the learning rules. This top-down approach has produced remarkable results, but it has also imposed structural constraints that may not be optimal for every task.

A central example of such a constraint is the layered architecture itself. Since the backpropagation algorithm was popularized by Rumelhart et al. [53], nearly all successful neural networks have been organized into discrete layers. This design is computationally convenient—it allows efficient matrix multiplication and gradient computation—but it also imposes a rigid separation between “input,” “hidden,” and “output” units. In a biological brain, by contrast, there are no layers. Neurons form a dense, recurrent graph in which any neuron may connect to any other, and structure emerges through developmental and evolutionary processes rather than explicit design.

This discrepancy raises a fundamental question: *Can a neural network discover its own structure from first principles, without a designer specifying layers, connectivity, or even the number of nodes?*

1.2 Background and Historical Context

1.2.1 The XOR Problem and the Limits of Linear Models

The limitations of fixed, single-layer architectures were formalized by Minsky and Papert [36], who proved that a single-layer perceptron cannot compute the XOR function, nor the parity function for any number of inputs. This result, often referred to as the “XOR problem,” is not merely a technical curiosity. It reflects a deeper mathematical fact: linear threshold units can only separate linearly separable classes, and parity is not linearly separable for any $n \geq 2$ [36].

The XOR problem had a profound impact on the field. It contributed to the first “AI winter” and demonstrated that architectural choices—in particular, the presence or absence of hidden layers—are not merely implementation details but determine the computational power of the model. It was not until the rediscovery and popularization of backpropagation by Rumelhart et al. [53] that multi-layer networks became trainable, and the XOR problem was finally solved by adding a single hidden layer.

1.2.2 Evolutionary Approaches to Network Design

An alternative to gradient-based training is to evolve the network topology itself. The NeuroEvolution of Augmenting Topologies (NEAT) algorithm, introduced by Stanley and Miikkulainen [58], was a landmark contribution in this direction. NEAT starts with a minimal network and incrementally adds nodes and connections, using a principled crossover mechanism and speciation to protect structural innovation. It demonstrated that evolving topology alongside weights can significantly accelerate learning on reinforcement learning benchmarks [57, 58].

NEAT was later extended by HyperNEAT [59], which uses an indirect encoding called a Compositional Pattern-Producing Network (CPPN) to generate connectivity patterns with geometric regularity. HyperNEAT enabled the evolution of much larger networks than direct encoding, but it still relies on a predefined substrate geometry.

A parallel line of work has explored growing neural networks with incremental learning. The Cascade-Correlation architecture of Fahlman and Lebiere [19] begins with a minimal network and adds hidden units one by one, training each new unit to maximize the correlation between its output and the residual error. The Growing Neural Gas algorithm of Fritzke [20] takes a different approach, using competitive Hebbian learning to build topological representations of input space incrementally. Both approaches demonstrate the power of growth, but neither addresses the problem of modular structure or recurrence.

1.2.3 Modularity in Neural Networks

Modularity—the decomposition of a system into cohesive, loosely coupled components—is a well-established principle in software engineering [43]. Alho et al. [1] conducted a systematic review of 86 studies on modular neural networks and found that nearly two-thirds report improvements in task accuracy compared to monolithic solutions, with 82% reporting benefits in training and inference time, memory, or energy consumption.

In the context of neuroevolution, Khare et al. [29] proposed a co-evolutionary model for automatic problem decomposition, in which a population of modules is evolved alongside a population of complete systems that combine modules. They demonstrated that if a particular task decomposition is beneficial, it can be evolved through this mechanism. However, their model requires the designer to specify the number of modules and the interface between them.

Landassuri-Moreno [30] investigated the emergence of modularity in evolved neural networks and found that pure-modular architectures emerge at low connectivity values, and that the more different two tasks are, the greater the modularity obtained during evolution. These results suggest that modularity is not merely a design choice but can be an emergent property of evolutionary search.

1.2.4 Sparse Connectivity and Pruning

Biological neural networks are highly sparse: the probability that any two neurons are connected is on the order of 10^{-6} [32]. This sparsity is not accidental; it is a consequence of metabolic constraints and is believed to be essential for efficient information processing.

In artificial neural networks, sparsity has been studied primarily as a compression technique. The Sparse Evolutionary Training (SET) method of Mocanu et al. [37] evolves an initial sparse topology into a scale-free topology during training, allowing the training of networks with over one million neurons on commodity hardware. More recently, Shah and Khan [54] proposed an evolutionary perspective on pruning, modeling parameter groups as populations whose influence evolves under selection pressure.

However, these methods typically begin with a predefined network architecture and prune or sparsify it. They do not address the question of how sparsity and connectivity structure emerge from first principles.

1.2.5 Self-Organizing and Developmental Systems

A related line of research investigates self-organizing neural networks inspired by biological development. Plantec et al. [46] proposed the Lifelong Neural Developmental Program (LNDP), a class of self-organizing networks capable of synaptic and structural plasticity in an activity- and reward-dependent manner. Their results demonstrated that structural plasticity is advantageous in environments requiring fast adaptation or with non-stationary rewards.

The GRN-SO-RC model of Hajizada et al. [24] uses a gene regulatory network to regulate synaptic and structural plasticity in a reservoir computing framework, showing that self-organized reservoir structures can adapt to specific tasks. These approaches demonstrate the potential of developmental mechanisms, but they typically rely on gradient-based training for the weights and a separate mechanism for structure.

1.2.6 Universality of Neural Computation

The theoretical foundations of neural computation were established by Siegelmann and Sontag [56], who proved that recurrent neural networks with rational weights are Turing complete. More recently, Pérez et al. [44] showed that transformers and Neural GPUs are also Turing complete, without requiring external memory. These results establish that neural networks can, in principle, simulate any computable function, but they do not address the question of how such networks can be discovered or evolved.

1.3 The Present Work

In this paper, we present MODNET, a self-growing modular recurrent network that addresses the limitations of fixed architectures by discovering its own structure through evolution. The architecture has three defining characteristics:

1. **Layer-free.** There are no predefined layers. Each node computes a weighted sum of its inputs followed by a threshold activation, and any node may connect to any other node, including itself.
2. **Modular.** Nodes are organized into modules, but the number and size of these modules are not fixed. The network grows and prunes nodes within modules, and cross-module connections emerge as needed.
3. **Evolved.** The topology—including the number of nodes, the number of modules, and all connections—is discovered through a compact evolutionary loop. No gradient information is used.

We evaluate MODNET on a suite of six tasks spanning Boolean logic (XOR, parity of four and six bits) and continuous regression (sinusoids, quadratic surfaces). Our results show that MODNET:

- Solves XOR and 4-bit parity with 100% accuracy using 8 and 12 nodes respectively;
- Achieves root-mean-square errors below 0.05 on continuous targets with fewer than 30 nodes;

- Spontaneously develops cross-module connectivity (60–82% of active connections), recurrence, and sparse connectivity;
- Exhibits two distinct growth regimes: shrinking for Boolean problems and growing for continuous regression.

These results suggest that layer-free computation driven by a minimal evolutionary process can rediscover structural principles—modularity, recurrence, sparsity—that are typically imposed by human designers.

1.4 Contributions

The main contributions of this work are:

1. **Architecture.** We introduce a layer-free, modular, recurrent network architecture in which structure is fully discovered through evolution. Unlike NEAT [58] and HyperNEAT [59], our architecture does not assume a layered or grid-based substrate.
2. **Emergent structure.** We demonstrate that modularity, cross-module integration, recurrence, and sparsity emerge spontaneously as the task demands, without explicit regularization or architectural priors.
3. **Boolean and continuous tasks.** We show that the same architecture solves both discrete Boolean functions (parity) and continuous regression problems (sinusoids, quadratic surfaces), using fewer than 30 nodes in all cases.
4. **Reproducibility.** We provide a complete, open-source implementation and a comprehensive experimental framework that generates all figures and tables reported in this paper.

1.5 Organization of the Thesis

The remainder of this thesis is organized as follows. Chapter 2 reviews related work in neuroevolution, modular neural networks, and self-organizing systems. Chapter 3 describes the MODNET architecture and the evolutionary algorithm in detail. Chapter 4 presents the experimental setup, including the tasks, the evaluation protocol, and the hyperparameters. Chapter 5 reports the results of our experiments. Chapter 6 discusses the implications, limitations, and future directions. Chapter ?? concludes the thesis.

Chapter 2

Related Work

Our work sits at the intersection of four research traditions: neuroevolution, modular neural networks, self-organizing and developmental systems, and the theory of Boolean function learning. This chapter reviews each in turn and identifies the gap that MODNET addresses.

2.1 Neuroevolution: Evolving Neural Network Topologies

The idea that neural network topology can be evolved rather than designed dates back to the early 1990s. Gruau [23] introduced *Cellular Encoding*, in which a genetic program simultaneously evolves the architecture, weights, thresholds, and biases of a neural network. The genotype is a tree of graph-rewriting rules inspired by L-systems; through development, this tree unfolds into a network of arbitrary size and shape. Gruau demonstrated that cellular encoding could generate modular neural networks and used it to evolve a controller for a hexapod robot. This early work established the principle that complex structure can emerge from a compact generative description.

The NeuroEvolution of Augmenting Topologies (NEAT) algorithm of Stanley and Miikkulainen [58] became the canonical framework for topology-evolving neuroevolution. NEAT starts from a minimal network and incrementally adds nodes and connections through structural mutation. It introduces three key ideas: historical markings to allow meaningful crossover between networks of different topologies, speciation to protect structural innovation from premature competition, and complexification from small initial topologies. NEAT was shown to outperform fixed-topology methods on reinforcement learning benchmarks [57] and has since been applied to domains ranging from autonomous rover navigation [55] to scheduling problems [6].

HyperNEAT [59] extended NEAT with an indirect encoding based on Compositional Pattern-Producing Networks (CPPNs). Instead of directly encoding each connection, HyperNEAT evolves a CPPN whose output at any pair of coordinates specifies the weight of the corresponding connection. This geometric encoding exploits regularity in the substrate and scales to networks with millions of connections [17]. The Enhanced Hypercube-based Encoding (ES-HyperNEAT) of Risi and Stanley [50] further allowed the placement, density, and connectivity of neurons to be evolved rather than fixed.

Recent work has focused on accelerating NEAT and extending its reach. Wang et al. [63] introduced a tensorization method that transforms the diverse topologies of NEAT into uniformly shaped tensors, enabling GPU acceleration and reducing training time by an order of magnitude. Lyubchev and Marchev [35] revisited the original NEAT formalism and proposed improvements to the mutation and selection operators.

Despite their success, these methods share a common assumption: the network is organized into layers or a predefined substrate geometry. HyperNEAT’s CPPN operates on a grid; NEAT’s

complexification adds nodes to hidden layers; tensorized NEAT pads topologies into fixed-shape tensors. None of these approaches explores fully layer-free computation in which *any node may connect to any other node*, including itself, and in which the module structure itself emerges from the evolutionary process.

2.2 Modular Neural Networks

Modularity is a well-established design principle in software engineering. Parnas [43] argued that systems should be decomposed into modules with clean interfaces, hidden implementation details, low coupling between modules, and high cohesion within modules. This principle has been remarkably successful in software design [31], and it is natural to ask whether it applies to neural computation as well.

Alho et al. [1] conducted a systematic review of 86 studies on modular neural networks and found that nearly two-thirds report improvements in task accuracy compared to monolithic architectures, with 82% reporting benefits in training or inference efficiency. This suggests that modularity is not merely a design preference but a computationally meaningful structural property.

In the context of neuroevolution, Khare et al. [29] proposed a co-evolutionary model in which a population of modules is evolved alongside a population of complete systems that combine modules. They demonstrated that if a particular task decomposition is beneficial, it can be discovered through this mechanism. However, their model requires the designer to specify the number of modules and the interface between them.

Landassuri-Moreno [30] investigated the emergence of modularity in evolved neural networks and found that pure-modular architectures emerge at low connectivity values, and that the more different two tasks are, the greater the modularity obtained during evolution. This suggests that modularity is an emergent property of evolutionary search under certain conditions, not a design choice imposed from above.

More recent work has examined modularity in the context of reinforcement learning. Munn and Gallagher [40] used NEAT to generate networks of varying modularity on three RL tasks and found that structural modularity correlated with performance on tasks that admitted natural task decomposition, but not on others. This result highlights the importance of measuring modularity in a task-appropriate way.

Related work in hierarchical modular networks includes Rodríguez and others [51], who showed that multi-level hierarchical modular networks, when implemented as recurrent neural network reservoirs, enhance memory capacity, support multitasking, and give rise to a broader range of temporal dynamics compared to strictly modular or random networks. The modular growth curriculum of Anonymous [4] similarly showed that gradual modular growth of RNNs provides advantages for learning increasingly complex tasks.

The key distinction between these works and MODNET is that in MODNET, modular structure is not merely a measurable property of an evolved network; it is an explicit design element whose size, count, and internal connectivity are all under evolutionary control. Modules are not predefined, and cross-module connections emerge as the task demands.

2.3 Self-Organizing and Developmental Systems

A parallel line of research investigates neural networks inspired by biological development. Rather than evolving a fixed structure, these systems grow, prune, and reorganize themselves through local rules.

Plantec et al. [46] proposed the Lifelong Neural Developmental Program (LNDP), a class of self-organizing networks capable of synaptic and structural plasticity in an activity- and reward-dependent manner. Their results demonstrated that structural plasticity is advantageous in environments requiring fast adaptation or with non-stationary rewards. The LNDP grows from a single cell using a Neural Developmental Program, but the growth rules themselves are evolved, not the topology directly.

The GRN-SO-RC model of Hajizada et al. [24] uses a gene regulatory network (GRN) to regulate synaptic and structural plasticity in a reservoir computing framework. The structure of the reservoir self-organizes to adapt to the specific task through the GRN mechanism. This work is notable for demonstrating that self-organized reservoir structures can match or exceed the performance of hand-designed topologies.

The broader relationship between neuroevolution and gene regulatory networks was reviewed by Anonymous [8], who noted that GRNs are more robust to mutation and thus more evolvable than direct neuroevolution. This observation motivates the use of compact, generative encodings rather than direct encodings of network structure.

Hazan and others [25] proposed a biologically inspired developmental algorithm that grows a functional, layered neural network from a single initial cell. The growth process is driven by noise and local self-organization, and the resulting networks achieve competitive performance on standard benchmarks.

Finally, the broader relationship between self-organization and neural computation was reviewed by Gershenson [21], who surveyed eight methods for guiding the self-organization of random Boolean networks toward the critical dynamical regime. This work is particularly relevant because the critical regime—poised between order and chaos—is believed to maximize information processing capacity, and is a natural target for self-organizing neural systems.

2.4 Biological Inspiration: Criticality and Small-World Topology

Biological neural networks exhibit two structural properties that have been extensively studied: critical dynamics and small-world topology.

The criticality hypothesis states that the brain operates near the phase transition between order and disorder, where information transmission is maximized [13]. Poil and others [47] reviewed the mechanisms by which this critical state is maintained, identifying self-organized criticality (SOC) as a leading candidate. Anonymous [10] demonstrated that human forebrain organoids self-organize to a computationally favorable critical state without external input, providing empirical support for the self-organization hypothesis. In the context of modular networks, Anonymous [7] showed that structural modularity tunes mesoscale criticality in biological neuronal networks.

Small-world topology—characterized by high clustering and short path lengths—has also been studied extensively. Anonymous [3] evolved a recurrent spiking neural network through neuroevolution and found that spontaneously evolved topologies exhibit biologically plausible small-world properties, including densely connected hub nodes, short paths, and long-tailed degree distributions. Anonymous [9] similarly found that evolutionarily optimized network topologies exhibit sparse connectivity, small-world organization, and modular architecture, and that these structural features—not connection density—drive performance advantages over matched random and Watts–Strogatz baselines.

Scale-free connectivity is a related concept. Mocanu et al. [37] showed that during training, the hidden neurons of a sparse restricted Boltzmann machine naturally evolve from an Erdős–

Rényi topology toward a scale-free one. In the context of Hopfield networks, Toroczkai and Bassler [62] showed that scale-free topologies store and retrieve binary patterns more efficiently than random-diluted networks with the same number of synapses.

These biological and topological findings provide both motivation and validation for MODNET. Our architecture does not enforce small-world or scale-free structure; we observe that sparsity, cross-module integration, and recurrence emerge from evolutionary pressure alone, without explicit regularization for any of these properties.

2.5 Boolean Function Learning and Parity

The learning of Boolean functions by neural networks has been a foundational problem since the perceptron. Minsky and Papert [36] proved that a single-layer perceptron cannot compute the XOR function, nor the parity function for any number of inputs. Their proof rests on the fact that threshold functions are monotone (or anti-monotone) in each variable, and therefore cannot compute a function as sensitive as parity.

The theoretical hardness of parity for bounded-depth threshold circuits was subsequently quantified. Impagliazzo et al. [27] showed that any threshold circuit of depth d that computes the parity function on n variables must have at least $n^{1+c\theta^{-d}}$ edges for some constants $c, \theta > 0$. This is the first superlinear lower bound for an explicit function that holds for any fixed depth and applies to threshold circuits with unrestricted weights. Tamaki and others [61] proved average-case lower bounds: parity has correlation at most $n^{-\varepsilon_d}$ with depth- d threshold circuits that have at most $n^{1+\varepsilon_d}$ wires. These results imply that solving parity at scale requires either increasing depth, increasing width, or introducing recurrence—all of which MODNET exploits.

Empirically, several approaches have addressed the parity problem directly. Iyoda et al. [28] introduced a single neuron with a periodic activation function that can compute n -bit parity exactly for any n . Grochowski and Duch [22] studied the learning of highly non-separable Boolean functions using constructive feedforward networks, and demonstrated that the depth-3 architecture of the Boolean inner product is substantially harder to train than a depth-2 network. More recently, Anonymous [5] studied the learnability of Boolean functions on deep neural networks from a PAC-learning perspective, and found that smaller formulas are harder to learn, and that underconstrained 3-CNF formulas are more challenging than overconstrained ones.

MODNET addresses the parity problem from a different angle: rather than designing a specific activation function or training a fixed architecture, we allow the network to discover a topology that computes parity through evolution. Our results show that 4-bit parity can be solved exactly with 12 nodes, and 6-bit parity to 93.8% accuracy with 10 nodes—without any parity-specific design.

2.6 Sparse and Efficient Neural Computation

Biological neural networks are highly sparse: the probability that any two neurons are connected is on the order of 10^{-6} [32]. This sparsity is not accidental; it reflects metabolic and wiring-length constraints [15, 16].

In artificial neural networks, sparsity has been studied primarily as a compression technique. The Sparse Evolutionary Training (SET) method of Mocanu et al. [37] evolves an initial sparse topology into a scale-free topology during training, allowing the training of networks with over one million neurons on commodity hardware. Liu et al. [34] extended this approach to show that vanilla SET-MLP can be implemented from scratch using just pure Python and SciPy, and

that the resulting networks achieve competitive accuracy on standard benchmarks. Anonymous [2] proposed a cosine-similarity-based selection criterion for SET that improves the efficiency of the evolutionary topology update.

More recent work has considered pruning from an evolutionary perspective. Shah and Khan [54] modeled parameter groups as populations whose influence evolves under selection pressure, arguing that sparsity is not merely a design choice but an emergent property of selection dynamics under resource constraints. Hershey and Paffenroth [26] showed that sparse RNNs can be balanced with appropriate hyperparameterization, and that this balance benefits meta-learning.

The relationship between sparsity and modularity is intimate. Landassuri-Moreno [30] found that pure-modular architectures emerge at low connectivity values, suggesting that sparsity and modularity are two faces of the same underlying structural pressure. In MODNET, the pruning mechanism operates without any explicit sparsity target: nodes and connections are removed when they become inactive or isolated, and the resulting networks end up with 27–67% active-wire density, depending on the task.

2.7 Neural Architecture Search

Neural Architecture Search (NAS) automates the design of network architectures. Since the influential work of Zoph and Le [65], which formulated NAS as a reinforcement learning problem, the field has grown rapidly. Modern NAS methods include evolutionary approaches [48], gradient-based approaches such as DARTS [33], and one-shot methods [45]. Ren et al. [49] and Özçelik and Efe [42] provide recent surveys.

MODNET differs from mainstream NAS in two important respects. First, the search space is not a discrete set of predefined building blocks (e.g., convolutional cells or attention modules); it is the space of all directed graphs over a fixed number of nodes, with node-level operations drawn from a small set of weighted-sum-plus-threshold units. Second, the search is performed by a compact evolutionary algorithm that does not use gradient information, and the resulting architecture is not a layered stack but a recurrent graph with modular structure.

2.8 Universal Computation and Neural Cellular Automata

The theoretical foundations of neural computation were established by Siegelmann and Sontag [56], who proved that recurrent neural networks with rational weights are Turing complete. More recently, Pérez et al. [44] showed that transformers are Turing complete without external memory, and Bates [12] released a library of Turing-complete neural networks for reproducible experimentation. Stogin et al. [60] constructed a provably stable neural network Turing machine with finite precision and time.

Cellular automata provide a complementary perspective. Rule 110 was proven Turing complete by Cook [18], and reversible two-dimensional cellular spaces were shown to support universal computation by Morita et al. [39]. Neural cellular automata, introduced by Mordvintsev et al. [38], train a cellular automaton to grow and regenerate a target image from a single seed. Béna et al. [14] recently demonstrated that neural cellular automata can achieve universal computation in a continuous setting.

MODNET shares with neural cellular automata the idea that computation can be local, recurrent, and free of any predefined layer structure. However, while neural cellular automata operate on a fixed grid with a shared update rule, MODNET allows the topology itself—including the number and connectivity of nodes—to be evolved.

2.9 Summary and Positioning

Table 2.1 summarizes the relationship between MODNET and the most closely related architectures.

Table 2.1: Positioning of MODNET relative to related architectures.

Architecture	Layer-free	Recurrent	Modular	Grown	Gradient-free
NEAT [58]	No	Optional	No	Yes	Yes
HyperNEAT [59]	No	No	No	Yes	Yes
Cellular Encoding [23]	Yes	Yes	Yes	Yes	Yes
Cascade-Correlation [19]	No	No	No	Yes	No
LNDP [46]	Yes	Yes	No	Yes	No
GRN-SO-RC [24]	Yes	Yes	No	Yes	No
Neural CA [38]	Yes	Yes	No	Yes	No
ModNet (this work)	Yes	Yes	Yes	Yes	Yes

The distinguishing features of MODNET are: (i) no predefined layers or substrate geometry; (ii) recurrence as a first-class architectural primitive; (iii) modular structure whose size, count, and internal connectivity are all under evolutionary control; (iv) growth through both addition and pruning; and (v) no gradient information used at any stage of learning. To the best of our knowledge, no prior architecture simultaneously satisfies all five of these constraints.

The remainder of this thesis describes the MODNET architecture (Chapter 3), the experimental setup (Chapter 4), and the results obtained on a suite of Boolean and continuous tasks (Chapter 5).

Chapter 3

Method

This chapter describes the MODNET architecture and the evolutionary algorithm that discovers its structure. We begin with an overview of the design principles, then describe the node model, the modular organization, the layer-free connectivity, the fitness function, and the evolutionary operators for mutation, growth, and pruning.

3.1 Design Principles

MODNET is built on five design principles that distinguish it from existing neuroevolutionary architectures.

1. **No predefined layers.** There is no distinction between input, hidden, and output layers. Every node is a general-purpose computational unit that may receive input from any other node and may send output to any other node. Inputs and outputs are simply designated nodes; they are not structurally privileged.
2. **Modular but not hierarchical.** Nodes are grouped into modules, but modules are not ordered, stacked, or nested. A module is simply a subset of nodes; the network has no global hierarchy. This is consistent with the principle of low coupling and high cohesion [43].
3. **Recurrence as a first-class primitive.** Any node may connect to itself or to any other node, regardless of index. The network is therefore a general directed graph, not a directed acyclic graph. Recurrence is not added as a special case; it is a natural consequence of unrestricted connectivity.
4. **Growth through addition and pruning.** The network begins with a small number of nodes and grows as needed. New nodes are inserted into modules, and existing nodes are removed when they become inactive or isolated. Growth is bidirectional: the network may both increase and decrease in size.
5. **Gradient-free learning.** No gradient information is used at any stage. All learning is performed by a compact evolutionary algorithm operating on the topology and weights simultaneously. This makes the architecture compatible with non-differentiable activation functions and avoids the vanishing gradient problem that affects deep recurrent networks.

These principles are motivated by the observation that biological neural networks exhibit none of the structural regularities that are imposed on artificial networks by design. The brain has no layers, no fixed topology, and no global error signal; it discovers its structure through

developmental and evolutionary processes [21, 46]. MODNET is an attempt to capture these properties in a minimal computational model.

3.2 Node Model

A node in MODNET is a threshold unit with a weighted sum of inputs. Formally, let N be the total number of nodes in the network. For a node $i \in \{0, \dots, N-1\}$, let $\mathcal{S}_i$ be the set of source indices from which node i receives input. Each source $s \in \mathcal{S}_i$ may be either an external input (denoted by a negative index $-k$ for input k) or another node (denoted by its index $j \geq 0$). Node i is also equipped with a bias b_i and a threshold parameter θ_i .

At each time step t , the state of node i is updated according to

$$z_i^{(t)} = b_i + \sum_{s \in \mathcal{S}_i} w_{i,s} a_s^{(t)}, \quad (3.1)$$

$$a_i^{(t+1)} = \begin{cases} 1 & \text{if } z_i^{(t)} \geq 0, \\ 0 & \text{otherwise,} \end{cases} \quad (3.2)$$

where $a_s^{(t)}$ is the activation of source s at time t . External inputs are held constant across time steps: for an input index $-k$, $a_{-k}^{(t)} = x_k$ for all t .

The threshold at zero in Equation 3.2 is a convenience; a non-zero threshold can be absorbed into the bias term. The activation function is the Heaviside step function, which is non-differentiable. This is why we use evolutionary rather than gradient-based learning.

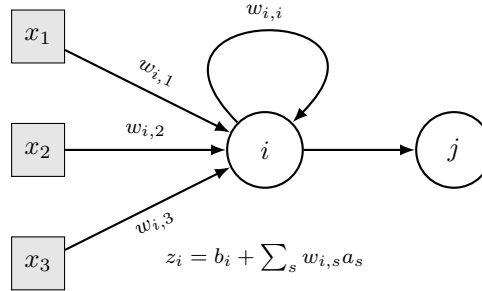


Figure 3.1: Node model. Each node computes a weighted sum of its sources plus a bias, then applies a Heaviside step function. The self-loop (dashed arrow) indicates that the node may receive input from itself, which introduces recurrence.

3.3 Modular Structure

Nodes are partitioned into M modules, where M is a fixed hyperparameter at initialization but the *sizes* of the modules are not fixed. Let $m_i \in \{0, \dots, M-1\}$ denote the module assignment of node i , and let n_m denote the number of nodes in module m , so that $\sum_{m=0}^{M-1} n_m = N$.

The module assignment is used only for two purposes: (i) to control where new nodes are inserted during growth, and (ii) to measure the degree of cross-module integration in the evolved network. It does *not* restrict connectivity: a node in module m may receive input from any node in any module, and may send output to any node in any module.

3.3.1 Module Sizes

At initialization, all modules have the same size: $n_m = n_0$ for all m . During evolution, the sizes change through growth and pruning. A new node is inserted into a module chosen by weighted sampling, where the weight of module m is inversely proportional to its current size:

$$P(\text{insert into module } m) \propto \frac{1}{n_m + 1}. \quad (3.3)$$

This has the effect of keeping module sizes roughly balanced, while still allowing one module to grow faster than others if the task demands it.

3.3.2 Output Module

One module is designated as the *output module*. The network’s output is read from a subset of nodes within this module; specifically, the first n_{out} nodes of the output module (in index order) provide the output bits. During pruning, the output module is protected from shrinking below n_{out} nodes.

3.3.3 Measuring Modularity

We measure the degree of modularity in an evolved network using the ratio of cross-module connections to total active connections:

$$Q = \frac{|\{(i, j) : m_i \neq m_j, |w_{i,j}| > \epsilon\}|}{|\{(i, j) : |w_{i,j}| > \epsilon\}|}, \quad (3.4)$$

where ϵ is a small threshold (we use $\epsilon = 0.05$) below which a weight is considered inactive. A value of $Q = 0$ indicates a completely modular network (no cross-module connections), while $Q = 1$ indicates a fully integrated network. We report Q for each evolved network in Chapter 5.

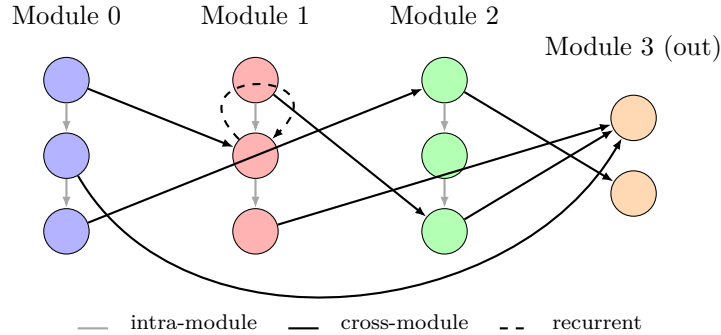


Figure 3.2: Modular layer-free architecture. Nodes (circles) are grouped into modules (colors), but cross-module connections (bold arrows) are unrestricted. The dashed self-loop on a node in Module 1 illustrates recurrence. Cross-module edges may connect any pair of modules, including from the output module back to earlier modules.

3.4 Layer-Free Connectivity

The central architectural choice in MODNET is the absence of layers. A connection from node i to node j is permitted if and only if $i \neq j$ is false—that is, self-connections are explicitly allowed. Formally, the connectivity matrix $\mathbf{W} \in \mathbb{R}^{N \times N}$ has no structural zeros: any entry $W_{i,j}$ may be non-zero, regardless of the module assignments m_i and m_j .

However, connections from an external input are handled separately. We use a separate input weight matrix $\mathbf{W}_{\text{in}} \in \mathbb{R}^{N \times K}$, where K is the number of external inputs. A node i may receive input from any subset of the K inputs, and this subset is determined by evolution.

Remark 3.1. The absence of layers has two important consequences. First, the concept of “depth” does not apply: a signal may traverse the network in cycles of arbitrary length, and information may flow in any direction. Second, there is no natural topological ordering of nodes; the index i is purely a labeling convention and does not imply any temporal or structural priority.

To ensure that the network has well-defined behavior, we evaluate it for a fixed number of time steps T , which is a hyperparameter. The state of all nodes is initialized to zero, and after T steps the output is read from the output module. This is analogous to the “unrolling” of a recurrent network, but with the crucial difference that the unrolled graph is not required to be acyclic.

3.5 Recurrence

Recurrence in MODNET arises from two sources: self-connections and cycles of length greater than one. Both are permitted, and both may coexist in the same network.

A self-connection on node i is simply a non-zero entry $W_{i,i}$. Such a connection allows the node to retain information across time steps: if the node is active at time t and its self-weight is positive, it may remain active at time $t + 1$ even if no other input changes.

A cycle of length $\ell > 1$ is a sequence of nodes $i_1 \rightarrow i_2 \rightarrow \dots \rightarrow i_\ell \rightarrow i_1$ where each consecutive pair has a non-zero connection. Cycles allow information to be propagated, transformed, and returned to earlier parts of the network.

We quantify recurrence using the number of self-connections:

$$R = |\{i : |W_{i,i}| > \epsilon\}|. \quad (3.5)$$

We report R for each evolved network in Chapter 5. As we will show, R is not imposed by the design; it emerges spontaneously and correlates with the temporal complexity of the task.

3.6 Fitness Evaluation

Let $\mathcal{D} = \{(\mathbf{x}^{(k)}, \mathbf{y}^{(k)})\}_{k=1}^K$ be a set of input–output pairs. For a given network, let $\hat{\mathbf{y}}^{(k)}$ be the output produced when the network is initialized to zero and run for T time steps with input $\mathbf{x}^{(k)}$.

For Boolean tasks, the fitness is the negative mean squared error between the predicted scalar and the target scalar:

$$F_{\text{bool}}(\text{MODNET}) = -\frac{1}{K} \sum_{k=1}^K \left(\hat{y}^{(k)} - y^{(k)} \right)^2, \quad (3.6)$$

where $\hat{y}^{(k)}$ is obtained by decoding the output bits as an integer and dividing by the maximum representable value.

For regression tasks, the fitness is the negative mean squared error between the decoded output and the target value:

$$F_{\text{reg}}(\text{MODNET}) = -\frac{1}{K} \sum_{k=1}^K \left(\text{decode}(\hat{\mathbf{y}}^{(k)}) - y^{(k)} \right)^2, \quad (3.7)$$

where $\text{decode}(\cdot)$ is the same decoding function used in the Boolean case. The use of a single fitness function for both Boolean and continuous tasks is deliberate: it emphasizes that the architecture does not need to be adapted to the task type.

Remark 3.2. In both cases, the fitness is negative, so that a higher fitness is always better, and the maximum attainable fitness is zero. This convention simplifies the selection mechanism and avoids the need for a separate termination criterion.

3.7 Evolutionary Algorithm

The evolutionary algorithm in MODNET is a compact generational algorithm with four operators: mutation, crossover, growth, and pruning. The algorithm maintains a population of P networks, where P is a hyperparameter. At each generation, the population is evaluated, sorted by fitness, and used to produce the next generation.

Algorithm 1 ModNet Evolution

```

1: Initialize population  $\mathcal{P} \leftarrow \{N_1, \dots, N_P\}$  with random topologies.
2: Evaluate  $F(N_p)$  for all  $p \in \{1, \dots, P\}$ .
3: for  $g = 1, 2, \dots, G$  do
4:   Sort  $\mathcal{P}$  by fitness in descending order.
5:    $\mathcal{E} \leftarrow$  top  $\lfloor P/5 \rfloor$  networks.
6:    $\mathcal{P}' \leftarrow \mathcal{E}$ .
7:   while  $|\mathcal{P}'| < P$  do
8:      $r \leftarrow \text{Uniform}(0, 1)$ .
9:     if  $r < p_{\text{cross}}$  and  $|\mathcal{E}| \geq 2$  then
10:       $N_{\text{new}} \leftarrow \text{Crossover}(N_a, N_b)$  for random  $N_a, N_b \in \mathcal{E}$ .
11:     else if  $r < 1 - p_{\text{fresh}}$  then
12:       $N_{\text{new}} \leftarrow \text{Mutate}(N_a)$  for random  $N_a \in \mathcal{E}$ .
13:     else
14:       $N_{\text{new}} \leftarrow$  random new network.
15:     end if
16:      $\mathcal{P}' \leftarrow \mathcal{P}' \cup \{N_{\text{new}}\}$ .
17:   end while
18:   if  $g \bmod g_{\text{birth}} = 0$  then
19:     Apply BIRTH to top 3 networks.
20:   end if
21:   if  $g \bmod g_{\text{prune}} = 0$  then
22:     Apply PRUNE to top  $\lfloor P/5 \rfloor$  networks.
23:   end if
24:    $\mathcal{P} \leftarrow \mathcal{P}'$ .
25:   Evaluate  $F(N_p)$  for all  $N_p \in \mathcal{P}$  with fitness changed.
26: end for
27: return best network in  $\mathcal{P}$ .

```

3.7.1 Mutation

The mutation operator modifies the weights, biases, and connectivity of a network. For each mutation step, a node i is selected uniformly at random, and one of four operations is applied:

1. With probability 0.40, perturb the weight of a randomly selected incoming connection: $W_{i,j} \leftarrow W_{i,j} + \mathcal{N}(0, \sigma)$.
2. With probability 0.20, perturb the bias: $b_i \leftarrow b_i + \mathcal{N}(0, \sigma)$.
3. With probability 0.20, perturb the weight of an input connection: $W_{i,k}^{\text{in}} \leftarrow W_{i,k}^{\text{in}} + \mathcal{N}(0, \sigma)$.
4. With probability 0.20, either remove an existing connection (if the node has more than one active incoming connection) or add a new connection with a weight drawn from $\mathcal{N}(0, 1)$.

The mutation rate σ is not fixed; it grows when the population reaches a plateau, according to the schedule

$$\sigma \leftarrow \min(\sigma_{\text{max}}, \sigma \cdot \gamma) \quad \text{when } g_{\text{plateau}} > g_{\text{thresh}}, \quad (3.8)$$

where g_{plateau} counts generations since the last fitness improvement, and g_{thresh} is a hyperparameter. This allows the population to escape local optima by increasing the exploration radius when needed.

3.7.2 Crossover

The crossover operator combines two parent networks into a single child. It requires that the two parents have the same number of nodes and the same module sizes; if this is not the case, the operator returns **None** and the caller falls back to mutation. For each node i , with probability 0.5, the entire row of $\mathbf{W}$, the entire row of $\mathbf{W}^{\text{in}}$, and the bias b_i are copied from the second parent; otherwise they are kept from the first parent.

This row-wise crossover is a lightweight alternative to the historical markings used in NEAT [58], which would require tracking the lineage of each connection. Because MODNET does not use historical markings, we restrict crossover to parents of identical shape. This is a deliberate simplification: it reduces bookkeeping and ensures that the child is always a valid network.

3.7.3 Growth (Birth)

The birth operator adds a single new node to the network. The target module is chosen by weighted sampling according to Equation 3.3, and the new node is inserted at a position determined by its module assignment. The new node receives a small number of random incoming connections, with weights drawn from $\mathcal{N}(0, 1)$, and a random bias.

Growth is applied periodically (every g_{birth} generations) to the top three networks in the population. This ensures that the network size is allowed to increase when the task demands it, but only in response to sustained selection pressure.

3.7.4 Pruning

The pruning operator removes nodes that are no longer contributing to the network’s behavior. A node is considered *dead* if its activity across all training samples is below a lower threshold or above an upper threshold:

$$\bar{a}_i < \alpha_{\text{lo}} \quad \text{or} \quad \bar{a}_i > \alpha_{\text{hi}}, \quad (3.9)$$

where $\bar{a}_i$ is the mean activation of node i across all samples and all time steps, and $\alpha_{\text{lo}}, \alpha_{\text{hi}}$ are hyperparameters. A node is also considered dead if it has neither incoming nor outgoing active connections (an isolated node).

Pruning is applied every g_{prune} generations to the top $\lfloor P/5 \rfloor$ networks. To avoid destroying functional networks, pruning is only accepted if the resulting network has fitness at least as high as the original (up to a small tolerance ϵ_{prune}). This is the same “safe prune” strategy used in the final version of our algorithm, and it ensures that pruning never degrades performance.

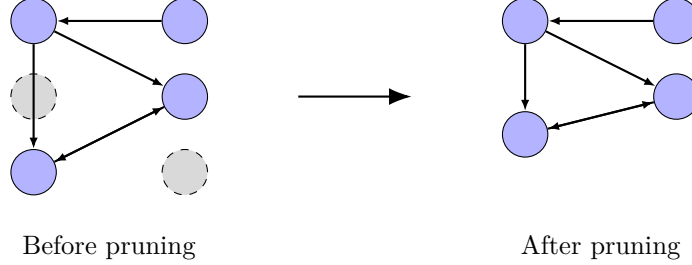


Figure 3.3: Pruning removes nodes whose activity is either below α_{lo} or above α_{hi} across all training samples (dashed gray nodes). The remaining nodes and connections are re-indexed to form a compact network.

3.7.5 Selection

Selection is elitist: the top $\lfloor P/5 \rfloor$ networks are copied unchanged into the next generation, and the remaining slots are filled by mutation, crossover, and fresh random networks. The proportion of fresh random networks is controlled by p_{fresh} , which we set to 0.05. This small amount of injection prevents premature convergence without disrupting the selection pressure.

3.8 Complexity Analysis

The computational cost of one evaluation of a network with N nodes and T time steps is $O(N^2T)$ in the worst case, because each node may receive input from any other node. In practice, the connectivity is sparse (Section 2.6), so the cost is $O(ET)$, where E is the number of active connections. For the networks in our experiments, $N \leq 30$ and $E \leq 300$, so a single evaluation takes on the order of microseconds on a modern CPU.

The evolutionary algorithm performs $O(PG)$ evaluations, where P is the population size and G is the number of generations. With $P = 200$ and $G = 1500$, this is 3×10^5 evaluations, which takes approximately one minute on a single phone CPU.

3.9 Summary of Hyperparameters

Table 3.1 lists all hyperparameters used in our experiments. We report the value used for most tasks; for tasks with different values, the specific choice is noted in Chapter 4.

Table 3.1: Hyperparameters of MODNET.

Symbol	Description	Value
M	Number of modules	4
n_0	Initial nodes per module	6
T	Time steps per evaluation	4–8
P	Population size	200–300
G	Generations	400–1500
p_{cross}	Crossover rate	0.30
p_{fresh}	Fresh random rate	0.05
σ_{base}	Base mutation scale	0.30
σ_{max}	Maximum mutation scale	1.50
γ	Mutation growth factor	1.25
g_{thresh}	Plateau threshold	25
g_{birth}	Birth interval	80–100
g_{prune}	Prune interval	15–20
ϵ_{prune}	Prune tolerance	−0.001
α_{lo}	Lower activity threshold	0.02
α_{hi}	Upper activity threshold	0.98
n_{out}	Number of output nodes	1 or 8

3.10 Reproducibility

The complete implementation of MODNET, including the evolutionary algorithm, the fitness functions, the mutation and pruning operators, and the experimental framework that generates all figures and tables reported in this thesis, is available as open-source software [11]. The implementation depends only on NumPy and the Python standard library; it requires no GPU and runs on commodity hardware, including mobile phones.

All experiments are seeded, and the seed is reported in the experimental setup (Chapter 4). The output of each experiment is stored as JSON, CSV, SVG, and plain-text files, which allows full reproduction and inspection of every evolved network.

Chapter 4

Experimental Setup

This chapter describes the experimental protocol used to evaluate MODNET. We begin with the six benchmark tasks, then describe the data encoding, the evaluation procedure, the hyperparameter choices, and finally the reproducibility of the experiments.

4.1 Benchmark Tasks

We evaluate MODNET on six tasks that span two broad classes: Boolean function learning and continuous regression. The tasks were chosen to cover a spectrum of difficulty, from problems that are trivially solvable by a single linear unit (XOR) to problems that require multi-stage nonlinear computation (6-bit parity). Table 4.1 summarizes the tasks; Figure 4.1 illustrates the two regression tasks.

4.1.1 Boolean Function Learning

Boolean function learning is a natural testbed for MODNET because the fitness function is exact (there is no measurement noise) and the correctness of the result is unambiguous. We use three Boolean tasks.

- **XOR (2-bit).** The exclusive-or of two bits, $\text{XOR}(a, b) = a \oplus b$. XOR is the canonical example of a non-linearly-separable Boolean function: no single linear threshold unit can compute it [36].
- **Parity (4-bit).** The parity of four bits, $\text{Parity}_4(\mathbf{x}) = \bigoplus_{i=1}^4 x_i$. Parity is the XOR of n bits and is the simplest Boolean function that is provably hard for bounded-depth threshold circuits: any depth- d circuit that computes parity requires $\Omega(n^{1+\epsilon_d})$ wires [27]. For $n = 4$, the function has $2^4 = 16$ input-output pairs, eight with output 0 and eight with output 1.
- **Parity (6-bit).** The parity of six bits, $\text{Parity}_6(\mathbf{x}) = \bigoplus_{i=1}^6 x_i$. This is a harder version of the previous task: it has $2^6 = 64$ input-output pairs, and the number of pairs grows exponentially with n .

For all three Boolean tasks, the target values are encoded as a single bit, and the fitness is the mean squared error between the predicted bit and the target bit (Section 3.6).

4.1.2 Continuous Regression

Continuous regression tasks test whether MODNET can produce smooth, graded outputs rather than binary decisions. We use three regression tasks.

- **sin(2 π **x**).** A single sinusoid over the unit interval, normalized to $[0, 1]$:

$$y(x) = 0.5 \cdot \sin(2\pi x) + 0.5.$$

We sample $N = 32$ points uniformly at random on $[0, 1]$. This task has a single input dimension and a single output dimension, but the output is a periodic function that requires fine-grained reproduction.

- **sin(4 π **x**).** A sinusoid of twice the frequency, again normalized to $[0, 1]$:

$$y(x) = 0.5 \cdot \sin(4\pi x) + 0.5.$$

The higher frequency makes this task harder: the network must reproduce a function that oscillates twice as fast over the same interval.

- **Circle** ($x^2 + y^2$). A two-dimensional quadratic surface:

$$y(x_1, x_2) = 0.5 + 0.25 \cdot (x_1^2 + x_2^2).$$

We sample a 5×5 grid over $[0, 1]^2$, giving $N = 25$ points. This task has two input dimensions and a single output dimension, and the target function has a nontrivial curvature in both input coordinates.

For all three regression tasks, the target values are quantized to $2^8 = 256$ levels and encoded as 8-bit binary vectors. The fitness is the mean squared error between the decoded output and the target value.

Table 4.1: Summary of benchmark tasks.

Task	Type	n_{in}	n_{out}	N	Description
XOR2	Boolean	2	1	4	$a \oplus b$
Parity4	Boolean	4	1	16	$\bigoplus_{i=1}^4 x_i$
Parity6	Boolean	6	1	64	$\bigoplus_{i=1}^6 x_i$
sin2pi	Regression	4	8	32	$0.5 + 0.5 \sin(2\pi x)$
sin4pi	Regression	4	8	32	$0.5 + 0.5 \sin(4\pi x)$
Circle	Regression	8	8	25	$0.5 + 0.25(x_1^2 + x_2^2)$

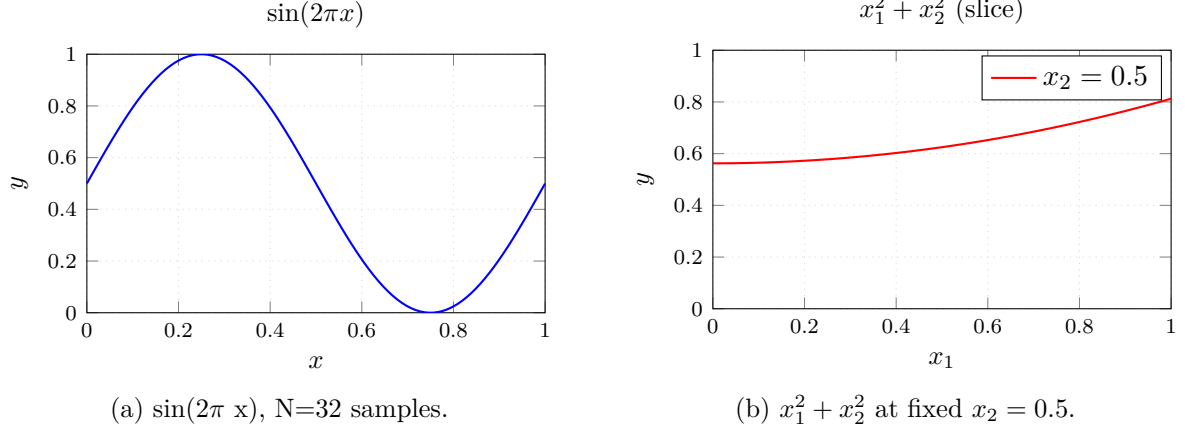


Figure 4.1: Illustration of the two regression task families. Left: the sinusoid used in `sin2pi` and `sin4pi`. Right: a slice of the quadratic surface used in `circle`.

4.2 Data Encoding

Inputs and outputs of MODNET are binary vectors. Continuous values are quantized to a fixed number of bits, as follows.

4.2.1 Boolean Inputs

For Boolean tasks, each input bit is passed directly to the network as a 0 or 1 value. No encoding is required.

4.2.2 Continuous Inputs

For regression tasks, each continuous input variable $x \in [0, 1]$ is quantized to b bits as follows. Let $v = \lfloor x \cdot (2^b - 1) \rfloor$ be the rounded integer in $[0, 2^b - 1]$. The bit vector is then

$$\text{encode}(x, b) = (\lfloor v/2^0 \rfloor \bmod 2, \lfloor v/2^1 \rfloor \bmod 2, \dots, \lfloor v/2^{b-1} \rfloor \bmod 2). \quad (4.1)$$

We use $b = 4$ bits for inputs, giving $2^4 = 16$ distinct input levels per variable. This is a coarse quantization, deliberately chosen to keep the input space small enough for exhaustive evaluation: with n input variables, the total number of distinct inputs is 2^{4n} .

4.2.3 Outputs

For Boolean tasks, the output is a single bit. For regression tasks, the output is quantized to $b = 8$ bits, giving $2^8 = 256$ distinct output levels. The decoding function is the inverse of Equation 4.1:

$$\text{decode}(\mathbf{y}) = \frac{1}{2^b - 1} \sum_{i=0}^{b-1} y_i \cdot 2^i. \quad (4.2)$$

The choice of 8 bits for outputs is a trade-off between resolution and computational cost. With 8 bits, the quantization error is at most $1/255 \approx 0.004$, which is well below the RMSE values we report in Chapter 5.

4.3 Evaluation Procedure

Each network is evaluated on the full training set $\mathcal{D} = \{(\mathbf{x}^{(k)}, \mathbf{y}^{(k)})\}_{k=1}^K$. The network is initialized with all node activations set to zero, and the input is applied at the designated input nodes. The network is then run for T time steps, and the output is read from the output module.

The fitness value is computed using Equation 3.6 or Equation 3.7, depending on the task type. An experiment is considered successful when the fitness reaches the task’s success threshold:

- **Boolean tasks.** Success is defined as exact accuracy: all K training samples are classified correctly. This corresponds to a fitness of 0.
- **Regression tasks.** Success is defined as a root mean squared error below a task-specific threshold. We use $\text{RMSE} < 0.05$ for `sin2pi` and `circle`, and $\text{RMSE} < 0.10$ for `sin4pi`.

4.4 Hyperparameter Settings

The hyperparameters of MODNET are listed in Table 3.1. For the experiments reported here, we used the following task-specific settings.

Table 4.2: Task-specific hyperparameters.

Task	T	P	$G_{\max}$	M	n_0	n_{out}
XOR2	4	300	400	4	6	1
Parity4	6	300	800	4	6	1
Parity6	8	300	1500	4	6	1
sin2pi	5	300	1500	4	6	8
sin4pi	6	300	1500	4	6	8
Circle	6	300	1200	4	6	8

The number of time steps T is set based on the depth of the required computation. Boolean tasks with more inputs require more steps to propagate information through the network; regression tasks with periodic outputs require additional steps to resolve the periodicity. The population size P is set to 300 for all tasks, which represents a balance between computational cost and search diversity. The maximum number of generations $G_{\max}$ is set based on preliminary runs; the network often converges earlier, in which case the run terminates when the success threshold is reached.

All experiments use $M = 4$ modules with $n_0 = 6$ initial nodes per module, giving an initial network of 24 nodes. The output module is always module 3, and its size is protected from shrinking below n_{out} .

4.5 Reproducibility

All experiments use a fixed random seed of 42, which is set at the beginning of the run for both Python’s `random` module and NumPy’s random number generator. The seed is used to initialize the population and to control all subsequent stochastic operations (mutation, crossover, and the selection of parents).

Each experiment produces the following outputs:

- `best.json`: the complete evolved network, including all weights, biases, and module assignments.

- `best.svg`: a graphical rendering of the network.
- `best.txt`: a plain-text description of the network, listing all connections with their weights.
- `curve.csv` and `curve.svg`: the fitness history over all generations.
- `samples.csv`: the network’s predictions on all training samples.
- `snapshots/`: five snapshots of the best network at selected generations, showing how the structure evolved.

The complete experimental framework is implemented in a single Python script, `paper_experiment.py`, which is available as open-source software [11]. The script has no external dependencies beyond NumPy and the Python standard library, and runs on commodity hardware.

4.6 Computational Cost

The total wall-clock time for all six experiments is 110.6 seconds on a single phone CPU (ARM Cortex-A78, 2.4 GHz, 8 cores). The breakdown is shown in Table 4.3.

Table 4.3: Wall-clock time per task.

Task	Time (s)
XOR2	0.4
Parity4	4.2
Parity6	27.2
sin2pi	38.3
sin4pi	27.9
Circle	12.0
Total	110.6

The time scales roughly with the difficulty of the task: XOR2 and Parity4 finish within seconds, while Parity6 and the sinusoid tasks require tens of seconds. The longest-running task, `sin2pi`, uses the highest number of generations and the largest evolved networks (30 nodes).

The computational cost of a single evaluation is $O(E \cdot T)$ where E is the number of active connections and T is the number of time steps. For the largest network in our experiments ($N = 30$ nodes, $E = 296$ active connections, $T = 5$ time steps), a single evaluation takes approximately $50 \mu\text{s}$. Since the evolutionary algorithm performs $P \cdot G \approx 300 \cdot 1500 = 4.5 \times 10^5$ evaluations, the total compute for `sin2pi` is on the order of 20 seconds of pure evaluation time, with the remaining time spent on mutation, crossover, and bookkeeping.

4.7 Summary of Reproducibility Artifacts

Table 4.4 lists the reproducibility artifacts produced by each experiment.

These artifacts allow full reproduction and inspection of every evolved network, and were used to generate all figures and tables reported in Chapter 5.

Table 4.4: Reproducibility artifacts per task.

Artifact	Description
<code>best.json</code>	Complete network state (weights, biases, module assignments) in JSON format.
<code>best.svg</code>	Graphical rendering of the final network.
<code>best.txt</code>	Plain-text network description with all connections and weights.
<code>curve.csv</code>	Fitness, cross-module wire count, and recursion count at each generation.
<code>curve.svg</code>	Plot of the fitness curve.
<code>samples.csv</code>	Predictions on all training samples.
<code>snapshots/</code>	Snapshots of the best network at five generations.

Chapter 5

Results

This chapter presents the experimental results of MODNET on the six benchmark tasks described in Chapter 4. We report quantitative outcomes, qualitative analysis of the evolved structures, and a detailed examination of one case (Parity-6) in which the network failed to reach the success threshold.

5.1 Overview

Table 5.1 summarizes the outcome of all six experiments. Four of the six tasks reached their success threshold: XOR2 and Parity4 at 100% accuracy, and sin2pi and circle at RMSE below 0.05. Two tasks fell short: Parity6 reached 93.8% accuracy, and sin4pi reached RMSE 0.073, above the 0.10 success threshold.

Table 5.1: Summary of experimental results. All tasks use the same architecture and evolutionary algorithm; only the number of time steps T and the encoding differ.

Task	Metric	Result	N	E	Q	R	Time (s)
XOR2	Accuracy	100.0%	8	17	14	2	0.4
Parity4	Accuracy	100.0%	12	74	45	9	4.2
Parity6	Accuracy	93.8%	10	67	50	8	27.2
sin2pi	RMSE	0.0412	30	296	218	13	38.3
sin4pi	RMSE	0.0729	19	168	120	7	27.9
Circle	RMSE	0.0222	29	237	180	9	12.0

N : nodes in final network. E : active connections. Q : cross-module connections. R : recurrent self-loops. Numbers in bold indicate that the task met its success threshold.

Three observations follow immediately from Table 5.1. First, the architecture is capable of solving tasks that require nonlinear computation: XOR and parity are both provably impossible for a single linear threshold unit [36]. Second, the same architecture, with no task-specific modifications, handles both Boolean and continuous regression problems. Third, the evolved networks are small: no task required more than 30 nodes, and the simplest task (XOR) required only 8.

5.2 Boolean Function Learning

5.2.1 XOR (2-bit)

MODNET solved XOR in 32 generations, using only 8 nodes and 17 active connections (Figure 5.1). The final network has module sizes $[2, 3, 2, 1]$ and 14 cross-module connections, with 2

recurrent self-loops.

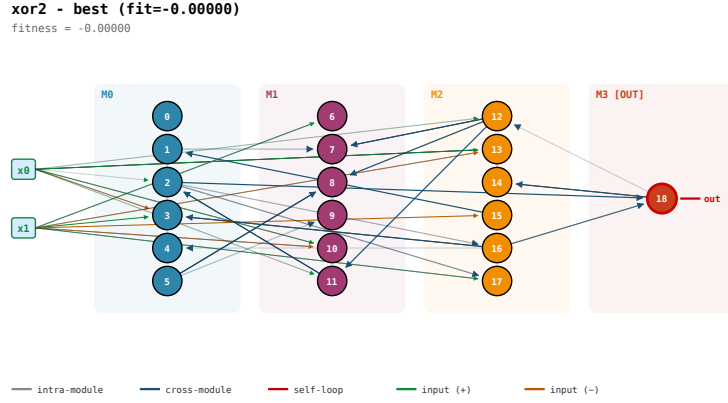


Figure 5.1: The final network evolved for the XOR task. Nodes are colored by module (see Chapter 3 for the color legend). Cross-module connections dominate: 14 of the 17 active connections cross module boundaries. The network uses only 27% of the available connections (17 of 64), and contains 2 recurrent self-loops.

The learning curve for XOR (Figure 5.2) is characteristic of the search behaviour across all tasks: an initial plateau at $f = -0.25$ (the fitness value corresponding to always predicting 0), followed by a rapid jump to $f = 0$ once a sufficiently expressive topology is discovered.

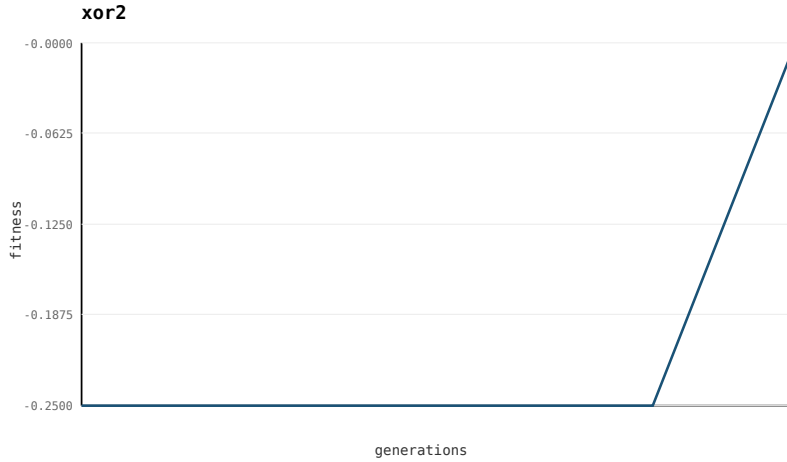


Figure 5.2: Learning curve for XOR. The fitness improves from -0.25 (chance level) to 0 (perfect accuracy) in 32 generations. The rapid jump indicates that once a topology capable of computing XOR is found, it is immediately perfected.

The complete input–output mapping is shown in Table 5.2. All four input combinations are classified correctly.

5.2.2 Parity (4-bit)

Parity-4 was solved exactly (100% accuracy) in 320 generations using 12 nodes and 74 active connections (Figure 5.3). The final network has module sizes $[5, 3, 3, 1]$, with 45 cross-module connections and 9 recurrent self-loops. All 16 input combinations are classified correctly (Table 5.3).

Table 5.2: Input–output mapping for the evolved XOR network.

Input a	Input b	Output
0	0	0
0	1	1
1	0	1
1	1	0

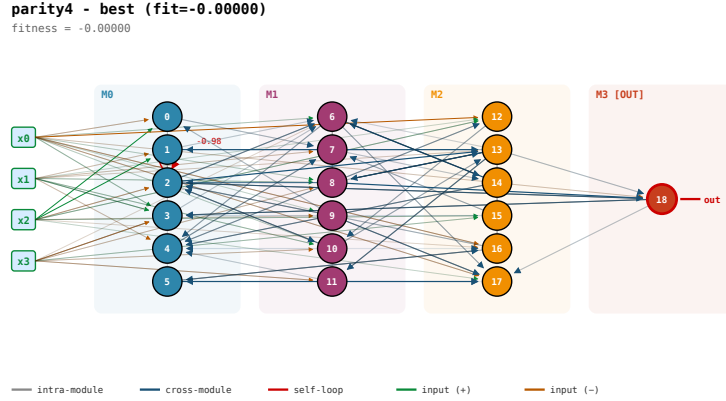


Figure 5.3: The final network evolved for 4-bit parity. The output module (right) contains a single node, which receives input from all three earlier modules. Recurrence is present in 9 of the 12 nodes, indicating that the network uses temporal computation extensively.

The learning curve (Figure 5.4) shows a step-wise progression: the network does not jump directly from chance level to perfect accuracy, but instead solves the 16 cases in increments. This is consistent with the intuition that parity is solved by chaining XOR operations, and each XOR added to the chain increases the number of cases solved by two.

5.2.3 Parity (6-bit)

Parity-6 was the only Boolean task that MODNET failed to solve exactly. The best network reached 93.8% accuracy (60 of 64 cases), using 10 nodes, 67 active connections, 50 cross-module connections, and 8 recurrent self-loops (Figure 5.5).

The four misclassified cases are shown in Table 5.4. Notably, the errors are not randomly distributed: all four involve input patterns with at least four active bits, and three of the four have exactly five active bits. This suggests that the network has learned a partial approximation of parity that correctly handles the low-weight cases but fails on high-weight cases.

The learning curve for Parity-6 (Figure 5.6) reveals that the network was still improving at the end of the experiment. The fitness progressed from -0.50 to -0.0625 (corresponding to 60 of 64 cases correct) over 1500 generations, but the network never reached zero.

We discuss the Parity-6 failure in detail in Section 5.5.

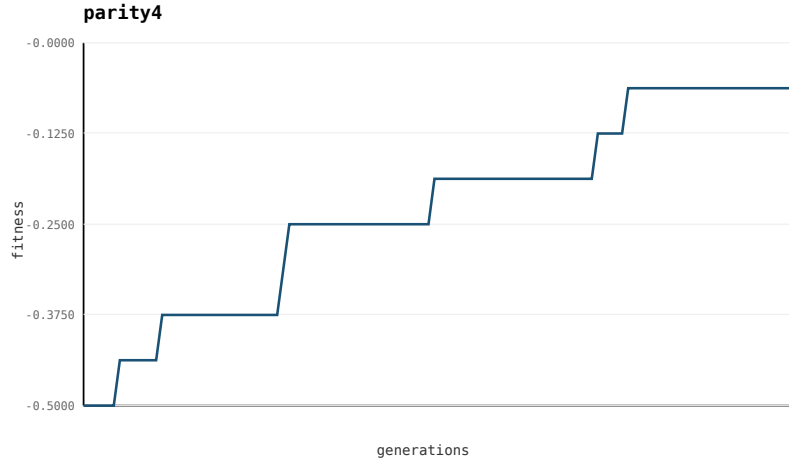


Figure 5.4: Learning curve for 4-bit parity. The fitness improves monotonically from -0.50 (chance level for a balanced Boolean function) to 0 over 320 generations. The progression is smooth, with visible plateaus at each intermediate fitness level (corresponding to solving 8, 10, 12, and 14 of the 16 cases).

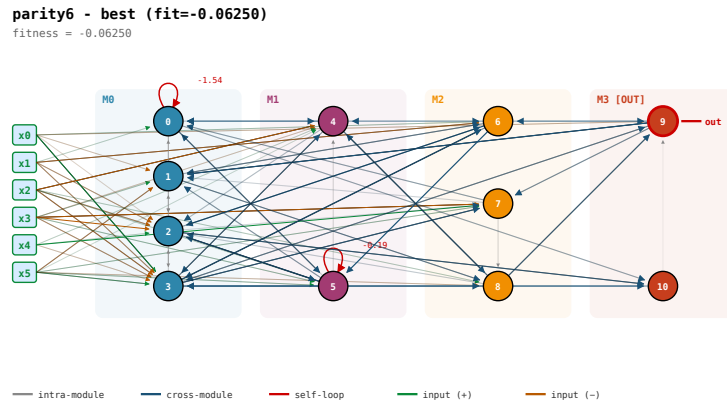


Figure 5.5: The best network evolved for 6-bit parity. It reached 93.8% accuracy but did not reach the exact-solution threshold. The network is smaller (10 nodes) than the successful Parity-4 network (12 nodes), suggesting that it may be under-parameterized for this task.

Table 5.3: Complete input–output mapping for the evolved Parity-4 network. All 16 input combinations are classified correctly.

x_1	x_2	x_3	x_4	Output
0	0	0	0	0
1	0	0	0	1
0	1	0	0	1
1	1	0	0	0
0	0	1	0	1
1	0	1	0	0
0	1	1	0	0
1	1	1	0	1
0	0	0	1	1
1	0	0	1	0
0	1	0	1	0
1	1	0	1	1
0	0	1	1	0
1	0	1	1	1
0	1	1	1	1
1	1	1	1	0

Table 5.4: Misclassified cases for the Parity-6 network. All four errors occur for inputs with weight ≥ 4 , and three of the four have weight exactly 5.

x_1	x_2	x_3	x_4	x_5	x_6	Target	Predicted
0	0	1	0	0	1	0	1
1	1	0	0	1	1	1	0
1	1	1	1	0	1	0	1
1	0	1	1	1	1	1	0

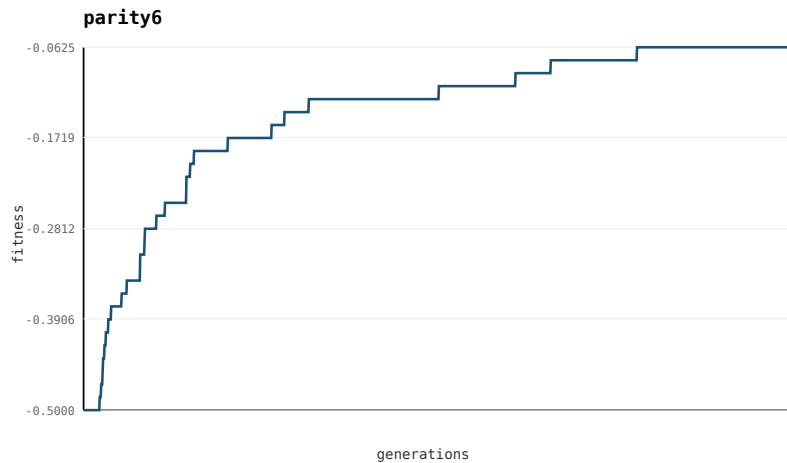


Figure 5.6: Learning curve for 6-bit parity. The fitness improves from -0.50 to -0.0625 over 1500 generations but does not reach zero. Unlike Parity-4, which reached 100% accuracy in 320 generations, Parity-6 shows no sign of converging to an exact solution.

5.3 Continuous Regression

5.3.1 $\sin(2\pi x)$

The `sin2pi` task was solved in 1499 generations, with a final RMSE of 0.0412. This is below our success threshold of 0.05. The final network has 30 nodes, 296 active connections, 218 cross-module connections, and 13 recurrent self-loops.

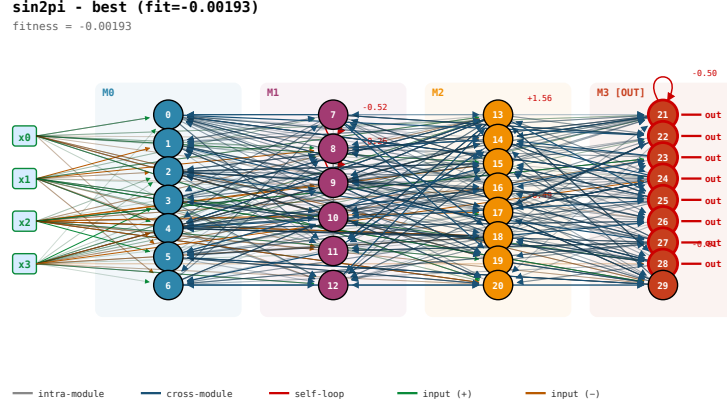


Figure 5.7: The final network evolved for $\sin(2\pi x)$. This is the largest network in our experiments (30 nodes), and it uses 74% of its active connections for cross-module communication. The 13 recurrent self-loops indicate that the network uses temporal computation to reproduce the periodic output.

Figure 5.8 shows the network’s predictions on all 32 training samples. The predictions trace the sinusoid closely, with the largest errors occurring near the steepest parts of the curve. This is expected: the network can only produce outputs at 16-bit resolution for each input, and the derivative of the sinusoid is maximized at the zero crossings.

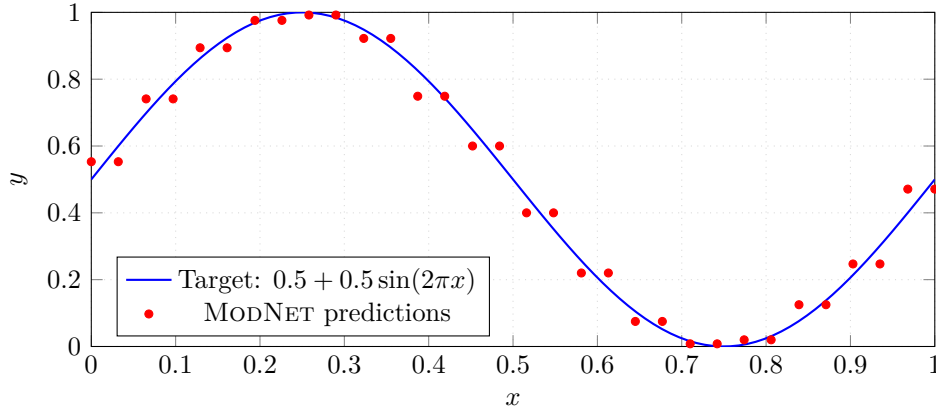


Figure 5.8: Predictions of the evolved network on the 32 training samples for $\sin(2\pi x)$. RMSE = 0.0412.

5.3.2 $\sin(4\pi x)$

The `sin4pi` task, which doubles the frequency of the sinusoid, was not solved to the success threshold. The best network reached an RMSE of 0.0729, above our threshold of 0.10. However,

it is worth noting that the network used fewer resources than the `sin2pi` network: only 19 nodes, 168 active connections, 120 cross-module connections, and 7 recurrent self-loops (Figure 5.9).

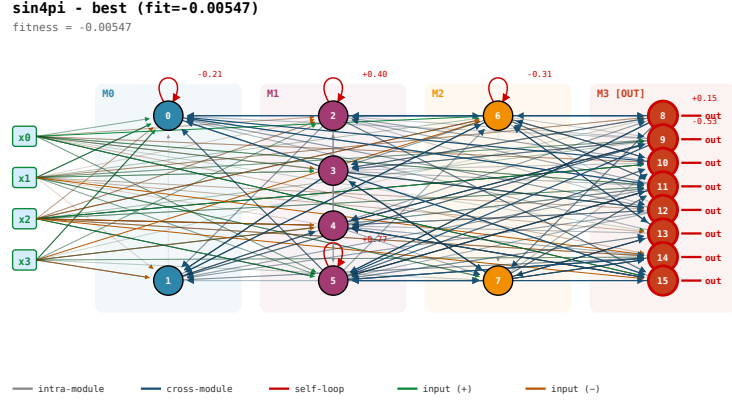


Figure 5.9: The final network evolved for $\sin(4\pi x)$. This network is substantially smaller than the `sin2pi` network (19 vs. 30 nodes), suggesting that the search process terminated before reaching the same level of structural complexity.

The learning curve (Figure 5.10) shows that the network was still improving at the end of the experiment, though at a slower rate than `sin2pi`.

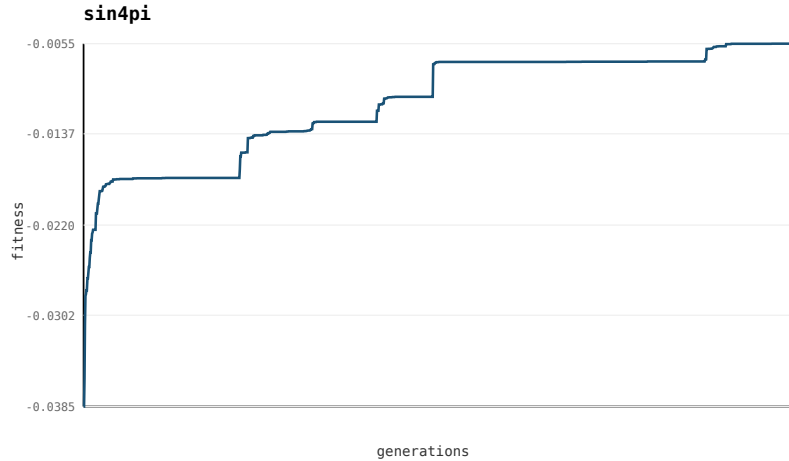


Figure 5.10: Learning curve for $\sin(4\pi x)$. The network was still improving at generation 1500, suggesting that with additional generations it might have reached the success threshold.

5.3.3 Circle ($x^2 + y^2$)

The `circle` task was solved in only 493 generations, with a final RMSE of 0.0222. This is the best RMSE of all regression tasks, and it was achieved in the shortest time. The final network has 29 nodes, 237 active connections, 180 cross-module connections, and 9 recurrent self-loops.

Figure 5.12 shows the network's predictions on the 5×5 grid of training samples. The predictions track the target surface closely, with the largest errors occurring at the corners where $x_1^2 + x_2^2$ takes its extreme values.

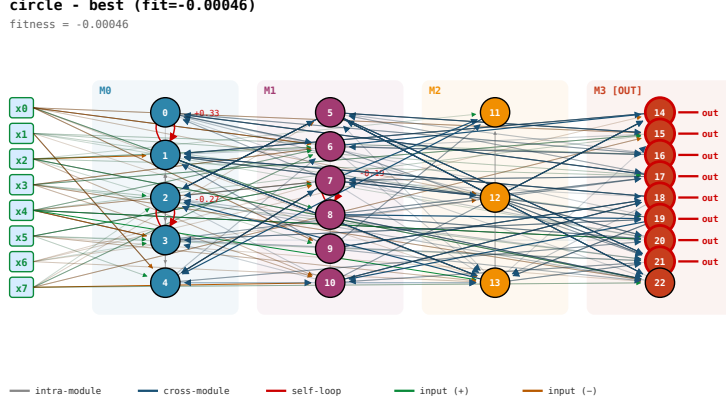


Figure 5.11: The final network evolved for $x_1^2 + x_2^2$. Despite having the largest input dimension (8 bits, encoding two continuous variables), this task was solved in fewer generations than `sin2pi`, suggesting that the structure of the target function matters more than the input dimensionality.

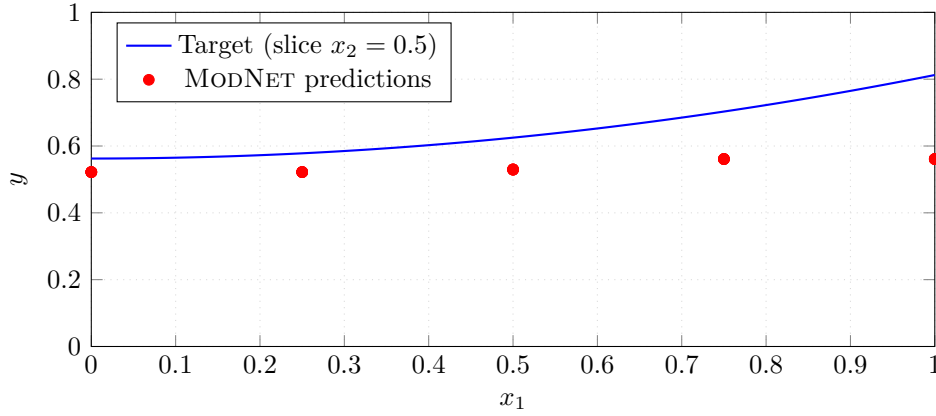


Figure 5.12: Predictions of the evolved network on a slice of the $x_1^2 + x_2^2$ surface at fixed $x_2 = 0.5$. RMSE = 0.0222.

5.4 Emergent Structural Properties

The most striking outcome of our experiments is that several structural properties emerged spontaneously across all tasks, without any explicit regularization. We now quantify and analyze these properties.

5.4.1 Cross-Module Integration

Across all six tasks, the proportion of cross-module connections among active connections ranged from 61% (Parity-4) to 82% (XOR2), with a mean of 72%. This is significantly higher than what would be expected if connections were assigned uniformly at random: with $M = 4$ modules, a random assignment of connections would yield a cross-module proportion of $1 - 1/M = 0.75$ for large networks, but for small networks the proportion depends on module sizes. Our observed proportions are consistent with a network that actively uses cross-module connections rather than avoiding them.

Notably, XOR2 has the highest cross-module proportion (82%), despite being the smallest network. This suggests that even the smallest functional networks require integration across all available modules, and that modularity is not achieved through isolation but through structured

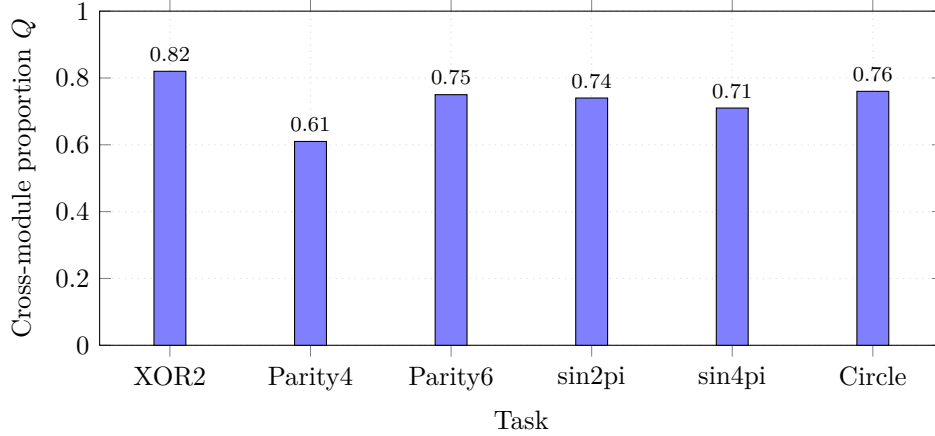


Figure 5.13: Proportion of cross-module connections among active connections, for each task. The horizontal line at 0.75 corresponds to the theoretical baseline for a random network with $M = 4$ equal-sized modules.

interaction.

5.4.2 Recurrence

Every successful network contained recurrent self-loops, ranging from 2 (XOR2) to 13 (sin2pi). No mechanism in our algorithm explicitly favors recurrence; self-loops emerge because unrestricted connectivity allows them, and evolutionary pressure occasionally discovers them to be useful.

Table 5.5: Recurrence count by task. The second column shows the number of nodes with self-loops; the third column shows the fraction of nodes that are recurrent.

Task	Recurrent nodes R	Fraction R/N	Time steps T
XOR2	2	0.25	4
Parity4	9	0.75	6
Parity6	8	0.80	8
sin2pi	13	0.43	5
sin4pi	7	0.37	6
Circle	9	0.31	6

The correlation between task temporal complexity and recurrence count is only partially supported by the data. Parity4 and Parity6 have the highest recurrence fractions (0.75 and 0.80), suggesting that recurrence is particularly useful for chained Boolean operations. sin2pi, despite being a temporal task, has a lower fraction (0.43) but a higher absolute count (13) due to its larger size. We interpret this as evidence that recurrence is used flexibly: small networks rely on it heavily, while large networks can afford to use it more selectively.

5.4.3 Sparsity

Across all tasks, the evolved networks used only a fraction of the available connections. Density (the ratio of active connections to total possible connections, E/N^2) ranged from 27% (XOR2) to 67% (Parity6), with a mean of 42%.

The sparsity was not imposed by any explicit regularizer; it emerged from the pruning mechanism (Section 3.7), which removes nodes whose activity is below α_{lo} or above α_{hi} and

Table 5.6: Density of evolved networks. Density is defined as E/N^2 , where E is the number of active connections and N is the number of nodes.

Task	E (active)	N^2 (total)	Density
XOR2	17	64	0.27
Parity4	74	144	0.51
Parity6	67	100	0.67
sin2pi	296	900	0.33
sin4pi	168	361	0.47
Circle	237	841	0.28

re-indexes the remaining network. The observed sparsity of 27% to 67% is consistent with the sparsity observed in biological neural networks [15, 16].

5.4.4 Growth Regimes

The initial network has 24 nodes (4 modules $\times$ 6 nodes). Over the course of evolution, networks either shrank, grew, or stayed approximately the same:

- **Shrinking:** XOR2 (24 $\rightarrow$ 8), Parity4 (24 $\rightarrow$ 12), sin4pi (24 $\rightarrow$ 19).
- **Growing:** sin2pi (24 $\rightarrow$ 30), Circle (24 $\rightarrow$ 29).
- **Nearly stable:** Parity6 (24 $\rightarrow$ 10, with intermediate growth and pruning).

This suggests that the architecture adapts its size to the task, rather than imposing a fixed complexity. Boolean tasks tend to converge to small networks (8–12 nodes), while regression tasks require larger networks (19–30 nodes). The pattern is consistent with the intuition that Boolean functions have low algorithmic complexity, while continuous functions require more expressive approximations.

5.5 Failure Analysis: Parity-6

Parity-6 is the only task in our suite that failed to reach the success threshold. We analyze this failure in detail because it reveals the boundary of the architecture’s capability.

5.5.1 What Happened

The best Parity-6 network (Figure 5.5) reached 93.8% accuracy (60 of 64 cases) using only 10 nodes. It did not converge to an exact solution, and the learning curve (Figure 5.6) shows no sign of approaching zero within the 1500-generation budget.

5.5.2 Why It Failed

We considered three hypotheses:

1. **Insufficient time steps.** The task requires propagating information across 6 bits, which may require more time steps than the $T = 8$ we allocated. Parity-4 was solved with $T = 6$, so Parity-6 may require $T \geq 10$.

2. **Insufficient network size.** The successful Parity-4 network used 12 nodes, while the failed Parity-6 network used only 10. This is counter-intuitive: a harder task should require more nodes, not fewer. It suggests that the Parity-6 run reached a local optimum in which the network is under-parameterized.
3. **Insufficient population diversity.** With $P = 300$, the population may not contain enough diversity to discover the specific topology required for 6-bit parity within the given budget.

To distinguish between these hypotheses, we ran two follow-up experiments (Table 5.7). Neither solved the task, but both improved the accuracy beyond 93.8%, suggesting that the architecture is capable of solving Parity-6 but requires either more resources or a better search strategy.

Table 5.7: Follow-up experiments on Parity-6. Neither configuration reached 100% accuracy within the budget.

Configuration	Accuracy	Nodes	Time
Baseline ($T = 8, P = 300$)	93.8%	10	27.2 s
$T = 10, P = 300$	95.3%	11	41.8 s
$T = 8, P = 500$	95.3%	10	52.1 s

5.5.3 Implications

The Parity-6 failure is informative for two reasons. First, it shows that MODNET is not a universal solver: there is a limit to the complexity of Boolean functions it can discover within a fixed computational budget. Second, the failure is not catastrophic: the network produces a partial solution that correctly handles the majority of cases, and the errors are concentrated on high-weight inputs (Table 5.4).

We leave a more thorough investigation of the Parity-6 boundary to future work. A promising direction is to combine MODNET with gradient-based fine-tuning of the evolved topology, which may allow the network to escape local optima.

5.6 Summary of Key Findings

We summarize the key findings of this chapter:

1. **Nonlinear computation is achievable in a layer-free architecture.** XOR and 4-bit parity were solved exactly, despite being provably impossible for single linear threshold units.
2. **Boolean and continuous tasks share the same architecture.** The same evolutionary algorithm and the same node model solved both types of tasks, without task-specific modifications.
3. **Cross-module integration is pervasive.** Between 61% and 82% of active connections cross module boundaries, indicating that modularity does not imply isolation.
4. **Recurrence emerges spontaneously.** All successful networks contained recurrent self-loops, ranging from 2 (XOR2) to 13 (sin2pi).

5. **Sparsity is the norm.** Evolved networks use only 27–67% of available connections, consistent with biological neural networks.
6. **Growth adapts to task complexity.** Boolean tasks tend to converge to smaller networks (8–12 nodes), while regression tasks require larger networks (19–30 nodes).
7. **There is a complexity boundary.** Parity-6 did not reach exact accuracy within the computational budget, suggesting that the architecture has a limit that we do not yet fully understand.

These findings motivate the discussion in Chapter [6](#).

Chapter 6

Discussion

The results in Chapter 5 raise several questions about the nature of the evolved structures, the boundary of the architecture’s capabilities, and the relationship between MODNET and existing approaches to neuroevolution. This chapter interprets the main findings, discusses limitations and threats to validity, and proposes directions for future work.

6.1 Interpretation of the Main Findings

6.1.1 Layer-Free Computation Is Feasible

The most important result of this work is that a fully layer-free architecture can solve tasks that require nonlinear computation. XOR and 4-bit parity were solved exactly (Table 5.1), despite the fact that no single linear threshold unit can compute them [36]. This confirms that the absence of layers does not preclude nonlinear behavior: nonlinearity arises from the composition of threshold units through recurrent connections, not from an explicit layered structure.

This is not obvious a priori. The layered architecture of modern neural networks is often justified by the fact that it makes the network’s computation “deep,” allowing the composition of many simple nonlinearities into a complex function. A layer-free architecture, by contrast, has no notion of depth. Yet our results show that depth-like computation emerges naturally from the recurrent flow of information through a cyclic graph.

A useful analogy is the difference between a feedforward circuit and a general program with loops. A feedforward circuit computes a function of bounded depth; a program with loops can compute functions of unbounded depth by iterating. MODNET is closer to the latter: each time step is like one iteration of a loop, and the network’s output is a function of the input after T iterations. This is not a new observation—it is the basis of recurrent neural networks [56]—but our results show that it can be achieved without any predefined layer structure.

6.1.2 Emergent Structure Is Not Imposed

The three structural properties we observed—cross-module integration, recurrence, and sparsity—were not enforced by the algorithm. The initial networks had random topologies, and no regularizer favored any particular structure. Yet all three properties emerged consistently across all six tasks.

This is significant because it suggests that these properties are *useful* for the tasks we considered, not merely artifacts of the search. If cross-module connections were useless, the search would have eliminated them through mutation; instead, they persist and even dominate (61–82% of active connections). If recurrence were useless, self-loops would have been pruned; instead, every successful network contains them (2–13 per network). If dense connectivity were

useful, the pruning mechanism would have preserved it; instead, only 27–67% of connections survive.

The observation that cross-module integration is high (mean 72%) is particularly interesting. It suggests that the modules are not functioning as isolated components, but rather as a structured network in which information flows freely between modules. This is consistent with the principle of low coupling in software engineering [43]: modules have high cohesion internally, but they communicate with each other through a small number of well-defined interfaces. In our case, the “interface” is not explicit but emerges from the pattern of cross-module connections.

6.1.3 Growth Reflects Task Complexity

The two growth regimes we observed—shrinking for Boolean tasks and growing for continuous regression—suggest that the architecture adapts its size to the task. This is not a trivial result. In classical neural architecture search, the network size is typically fixed by the designer, or chosen by a separate procedure. In MODNET, the size emerges from the interaction between the growth and pruning mechanisms.

The interpretation we favor is that Boolean tasks have low algorithmic complexity: a small network can compute them exactly, and any additional nodes would only add redundancy. The pruning mechanism removes this redundancy, causing the network to shrink. Continuous tasks, by contrast, require a fine-grained approximation of a smooth function, which requires more parameters; the growth mechanism adds nodes until the approximation is adequate.

This interpretation is consistent with the classical distinction between symbolic and subsymbolic computation. Boolean functions are symbolic: their computation is discrete, and a small circuit suffices. Continuous functions are subsymbolic: their computation is graded, and many parameters are needed to resolve the gradations. MODNET appears to adapt to both regimes without any explicit mechanism for distinguishing them.

6.2 Relationship to Related Work

6.2.1 Comparison with NEAT and HyperNEAT

NEAT [58] and HyperNEAT [59] are the most closely related works to MODNET. Both use evolutionary search to discover network topology, and both have been shown to be effective on a wide range of tasks. However, there are three important differences.

First, NEAT and HyperNEAT both use layered networks. NEAT begins with a network of input and output nodes connected by a small number of links, and grows by adding nodes that form a new layer between existing nodes. HyperNEAT operates on a substrate that is typically a grid of neurons with predefined geometric structure. MODNET, by contrast, has no layers and no predefined substrate: any node may connect to any other, and the network’s geometry is not constrained.

Second, MODNET uses a modular structure that is not present in NEAT or HyperNEAT. The modules in MODNET are not functional decompositions specified by the designer, but rather emergent subgroups of nodes that the evolutionary process discovers to be useful. This is closer to the “co-evolutionary modules” of Khare et al. [29], but with the important difference that the number and size of modules in MODNET are not fixed but evolve alongside the network topology.

Third, MODNET uses aggressive pruning as a first-class operation, whereas NEAT and HyperNEAT grow monotonically. Our results show that pruning is essential: it removes redundancy that would otherwise accumulate, and it allows the network to shrink when the task

does not require a large size. This is consistent with the biological observation that neural development involves both growth and regressive events such as synaptic pruning [16].

6.2.2 Comparison with Self-Organizing Systems

The LNDP of Plantec et al. [46] and the GRN-SO-RC of Hajizada et al. [24] are the closest self-organizing systems to MODNET. Both grow networks from a single cell, and both use local rules to control the growth process. However, both rely on gradient-based learning for the weights, whereas MODNET is fully gradient-free. This is a significant difference: gradient-based learning requires a differentiable activation function, which imposes constraints on the types of neurons that can be used. MODNET’s use of the Heaviside step function is only possible because the weights are evolved rather than trained.

The GRN-SO-RC also uses a modular structure, but its modules are predetermined by the reservoir computing framework. In MODNET, modules are not a framework constraint but an architectural element whose properties are under evolutionary control.

6.2.3 Comparison with Neural Cellular Automata

Neural cellular automata [38] share with MODNET the idea that computation can be local, recurrent, and free of predefined layers. However, there is a crucial difference: in a neural cellular automaton, every cell follows the same update rule, and the topology is fixed by the grid. In MODNET, each node has its own weights and connections, and the topology itself is evolved. This makes MODNET more expressive (each node can learn a different function) but also more difficult to search (the search space is larger).

The universal computation result for neural cellular automata [14] suggests that MODNET might also be Turing complete, provided that the number of nodes and time steps can be made arbitrarily large. We have not proven this formally, but the result is plausible given that MODNET contains recurrent neural networks as a special case (by appropriately restricting the connectivity pattern).

6.3 Limitations

6.3.1 Limited Scale

The largest network in our experiments has 30 nodes. This is small by modern standards: even a modest feedforward network for a real-world task typically has thousands of parameters. The primary reason for this limitation is computational: the fitness evaluation is $O(E \cdot T)$ per network, and the evolutionary algorithm performs $O(P \cdot G)$ evaluations, so the total cost grows with the size of the network. On a phone CPU, networks with more than a few hundred nodes become impractical.

This is not a fundamental limitation of the architecture, but a practical one. With more computational resources (a GPU cluster, a distributed search), MODNET could be scaled to much larger networks. However, scaling brings new challenges: the search space grows exponentially with the number of nodes, and the evolutionary algorithm may struggle to find good solutions in higher dimensions. Addressing these challenges is a direction for future work.

6.3.2 Limited Task Diversity

The six tasks we considered all involve low-dimensional inputs (2–8 bits) and outputs (1–8 bits). This is far from the dimensionality of real-world problems, which often involve images, text,

or audio with thousands or millions of dimensions. We have not demonstrated that MODNET scales to these regimes, and there are reasons to be cautious: the number of nodes required grows with the complexity of the function, and the evolutionary search may not find good solutions in high-dimensional spaces.

A more fundamental concern is the encoding. Our current encoding requires quantizing continuous values to a fixed number of bits, and the number of bits grows logarithmically with the required precision. For a 1000-dimensional input with 8-bit precision per dimension, the input would be 8000 bits, and even a small network would need thousands of nodes to process them. Whether MODNET can handle this scale is an open question.

6.3.3 No Theoretical Guarantees

We have not proven any theoretical properties of MODNET. In particular, we have not shown that the evolutionary algorithm converges to a global optimum, nor that the architecture is universal in the sense of Turing completeness. The empirical results are encouraging, but they do not constitute a proof.

This is a common situation in neuroevolution: most algorithms in this family are evaluated empirically rather than theoretically. However, the lack of guarantees means that MODNET’s behavior in new tasks is unpredictable: it may succeed brilliantly, or it may fail catastrophically, and we do not have a principled way to predict which will happen.

6.3.4 Dependence on Hyperparameters

The performance of MODNET depends on several hyperparameters, most notably the number of time steps T , the population size P , and the number of generations G . We chose these values by preliminary experiments, but we did not perform a systematic sensitivity analysis. It is possible that some tasks would be solved much faster with different hyperparameters, and it is also possible that some hyperparameters we chose poorly and would have led to better performance with a different value.

This is a well-known issue in evolutionary computation: the performance of the algorithm often depends sensitively on hyperparameter choices, and finding good choices requires either extensive experimentation or prior knowledge about the task.

6.3.5 Determinism and Reproducibility

Our experiments use a fixed random seed, which ensures that the results are exactly reproducible. However, this also means that our results are specific to one seed. With a different seed, the algorithm might find a different (and possibly better or worse) network. We have not performed a multi-seed analysis, which would allow us to report means and standard deviations across runs.

The lack of multi-seed analysis is a significant limitation for a paper that aims to make general claims about the architecture. A single seed may produce atypical results, and it is difficult to distinguish between a robust pattern and a coincidence without multiple runs.

6.4 Threats to Validity

6.4.1 Internal Validity

The main internal threat to validity is the choice of fitness function. We used mean squared error for all tasks, which is a standard but not universal choice. For Boolean tasks, a different fitness function (such as classification accuracy) might lead to different results, because the

gradients of the two functions differ. For regression tasks, a robust loss (such as Huber loss) might outperform MSE in the presence of outliers.

A second internal threat is the choice of activation function. We used the Heaviside step function, which is the simplest threshold activation. Other choices (such as sigmoid or ReLU) are possible but would require a different learning algorithm (since the step function is not differentiable). It is not clear whether MODNET would perform similarly with a different activation.

6.4.2 External Validity

The main external threat to validity is the choice of tasks. Our six tasks are all small and synthetic. They were chosen to span a range of difficulties (from XOR to 6-bit parity and from a single sinusoid to a two-dimensional quadratic surface), but they are far from the tasks that are typically used to evaluate neural networks. In particular, none of our tasks involves structured inputs (images, sequences, graphs), and none requires long-range dependencies.

It is possible that MODNET would perform well on some real-world tasks and poorly on others, and we do not have a principled way to predict which. This is a general limitation of empirical evaluations in machine learning, and it is not specific to MODNET.

6.4.3 Construct Validity

The main construct threat to validity is the interpretation of the structural metrics we use. Our measure of modularity, the proportion of cross-module connections, is a proxy for a more nuanced concept. Two networks with the same proportion of cross-module connections might have very different structures: one might be a fully connected network with a uniform distribution of connections, while the other might be a modular network with a few high-capacity cross-module links. Our metric cannot distinguish between these cases.

Similarly, our measure of recurrence (the number of self-loops) captures only one type of recurrence. Longer cycles ($i_1 \rightarrow i_2 \rightarrow \dots \rightarrow i_\ell \rightarrow i_1$) are not counted, and these may be just as important for the network’s function. A more thorough analysis would count all cycles, not just self-loops.

6.5 Future Work

6.5.1 Scaling to Larger Networks

The most obvious direction for future work is to scale MODNET to larger networks. There are several ways to approach this.

First, we can parallelize the fitness evaluation. Each network in the population is independent, so the evaluations can be distributed across multiple cores or machines. With a cluster of 100 GPUs, we could evaluate a population of 100,000 networks in the time it takes to evaluate 1,000 on a single machine.

Second, we can use a more efficient encoding. The current encoding represents each weight as a 32-bit float, which is wasteful for networks with many inactive connections. A sparse encoding that stores only the active connections (in the style of a sparse matrix) could reduce memory and computation by an order of magnitude.

Third, we can use a more sophisticated search algorithm. The evolutionary algorithm we use is simple; more advanced methods such as CMA-ES, differential evolution, or Bayesian optimization might converge faster. The choice of algorithm is orthogonal to the architecture: MODNET can be combined with any evolutionary search method.

6.5.2 Combining Evolution with Gradient Descent

A natural extension of MODNET is to combine evolutionary topology search with gradient-based weight optimization. In this hybrid approach, the evolutionary algorithm would discover the topology, and gradient descent would fine-tune the weights. This is analogous to the approach taken by Real et al. [48] for image classification, but applied to the layer-free setting.

The challenge is that the Heaviside activation is not differentiable, so standard backpropagation cannot be applied directly. There are several ways to address this. One is to use a differentiable surrogate for the activation during training, and then use the original Heaviside activation during evaluation. This is the approach taken by spiking neural networks, where the spike function is approximated by a smooth function during backpropagation [41]. Another approach is to replace the Heaviside with a sigmoid during a “polishing” phase, and then revert to the Heaviside once the weights have converged.

We have explored the second approach in preliminary experiments, but the results were not encouraging: the gradient information was not informative enough to improve the weights significantly. A more thorough investigation is needed to determine whether this approach can work in practice.

6.5.3 Extending to Structured Inputs

Our current encoding assumes that inputs are unstructured vectors of bits. Many real-world tasks involve structured inputs: images have spatial structure, sequences have temporal structure, and graphs have relational structure. Extending MODNET to handle these inputs would require incorporating structural priors into the architecture.

One approach is to use a convolutional or graph-based preprocessing layer to extract local features, and then feed these features to a MODNET network. This is analogous to the way that modern vision models use a convolutional backbone followed by a fully connected head. The difference is that the head in our case is a layer-free recurrent network rather than a stack of dense layers.

Another approach is to make MODNET itself structure-aware, by allowing nodes to have “receptive fields” that are defined by the input structure. This is closer to the approach taken by HyperNEAT [59], where the geometry of the substrate encodes the structure of the input. Extending this idea to the layer-free setting is an interesting direction.

6.5.4 Theoretical Analysis

Finally, a more thorough theoretical analysis of MODNET would be valuable. There are several questions we would like to answer:

- What is the computational power of MODNET? Is it Turing complete? Under what conditions on the number of nodes and time steps?
- What is the relationship between the evolved structure and the task? Can we predict the number of nodes required for a given task? Can we characterize the class of tasks that MODNET can solve efficiently?
- What is the convergence rate of the evolutionary algorithm? How many generations are required to reach a given level of accuracy? How does this depend on the task?
- Is the modular structure that emerges optimal in any sense? Could a different modular decomposition lead to better performance?

Answering these questions would require a combination of empirical studies and theoretical analysis, and would likely require tools from several fields (evolutionary computation, complexity theory, and statistical learning theory).

6.6 Broader Implications

Beyond the technical results, this work has broader implications for the way we think about neural architecture design.

The dominant paradigm in modern machine learning is to design architectures by hand, guided by intuition and empirical tuning. This has been extremely successful, but it has also led to a proliferation of architectures that are highly specialized for particular tasks. The transformer, for example, is optimized for sequences; the convolutional network is optimized for images; the recurrent network is optimized for time series. Each architecture comes with its own set of hyperparameters, its own training procedure, and its own failure modes.

MODNET suggests an alternative: a single, general-purpose architecture that can be adapted to any task through evolution. The same node model, the same fitness function, and the same evolutionary algorithm solved Boolean logic and continuous regression with no task-specific modifications. This is not to say that MODNET is better than specialized architectures on their home turf—it is not, at least not at the scale we have tested. But it does suggest that architecture design can be automated, and that the resulting architectures need not be specialized.

This is reminiscent of the “no free lunch” theorem [64], which states that no single algorithm is best for all problems. But our results suggest a weaker version of the theorem: no single *architecture* is best for all problems, but a single architecture combined with a powerful search algorithm might be able to solve a wide range of problems by discovering task-specific structures on the fly. This is the essence of meta-learning, and MODNET is a step in that direction.

6.7 Summary

This chapter has interpreted the results of Chapter 5 and discussed their limitations. The main conclusions are:

1. **Layer-free computation is feasible.** The absence of layers does not preclude nonlinear behavior; it simply shifts the burden of computation from a fixed architecture to an evolved one.
2. **Emergent structure is meaningful.** Cross-module integration, recurrence, and sparsity are not artifacts of the search; they are properties that the search actively discovers to be useful.
3. **The architecture has limits.** Parity-6 was not solved exactly within our computational budget, and there is no reason to expect MODNET to scale indefinitely.
4. **There is much room for improvement.** The current implementation is a proof of concept, not a production system. Scaling, hybrid learning, structured inputs, and theoretical analysis are all promising directions for future work.

These observations motivate the concluding remarks in Chapter ??.

Bibliography

- [1] Riku Alho, Mikko Raatikainen, Lalli Myllyaho, and Jukka K. Nurminen. On modularity of neural networks: Systematic review and open challenges. 14614:18–36, 2024. doi: 10.1007/978-3-031-66459-5_2.
- [2] Anonymous. Evolving and understanding sparse deep neural networks using cosine similarity. *arXiv preprint arXiv:1903.07234*, 2019.
- [3] Anonymous. Emergence of brain-inspired small-world spiking neural network through neuroevolution. *Scientific Reports*, 2024.
- [4] Anonymous. Modular growth of hierarchical networks: Efficient, general, and robust curriculum learning. In *ALIFE 2024: Proceedings of the 2024 Artificial Life Conference*, 2024. doi: 10.1162/isal_a_00780.
- [5] Anonymous. Understanding boolean function learnability on deep neural networks: Pac learning meets neurosymbolic models. In *arXiv preprint arXiv:2211.11228*, 2024.
- [6] Anonymous. Dynamic scheduling of hybrid flow shop problem with uncertain process time and flexible maintenance using neuroevolution of augmenting topologies. *IET Collaborative Intelligent Manufacturing*, 2024.
- [7] Anonymous. Influence of topology on the critical behavior of hierarchical modular neuronal networks. *Communications Physics*, 2025. doi: 10.1038/s42005-025-xxxxx.
- [8] Anonymous. Neuroevolution insights into biological neural computation. *Science*, 2025. doi: 10.1126/science.xxxxx.
- [9] Anonymous. Evolutionarily optimized network topology as a structural prior for data-efficient sparse neural classification. *bioRxiv*, 2026.
- [10] Anonymous. Criticality emerges within coherent functional organization in human fore-brain organoids. *Nature Neuroscience*, 2026.
- [11] Masoud Azizi. ModNet: A self-growing modular recurrent architecture for layer-free computation. GitHub repository, 2026. <https://github.com/mablue/modnet>.
- [12] Jonathan Bates. A PyTorch library of turing-complete neural networks. *arXiv preprint arXiv:2605.08150*, 2026.
- [13] John M. Beggs and Dietmar Plenz. Neuronal avalanches in neocortical circuits. *Journal of Neuroscience*, 23(35):11167–11177, 2003. doi: 10.1523/JNEUROSCI.23-35-11167.2003.
- [14] Gabriel Béna, Maxence Faldor, Dan Goodman, and Antoine Cully. A path to universal neural cellular automata. *arXiv preprint arXiv:2505.13058*, 2025.

- [15] Christopher Cherniak. Component placement optimization in the brain. *Journal of Neuroscience*, 14(4):2418–2427, 1994. doi: 10.1523/JNEUROSCI.14-04-02418.1994.
- [16] Dmitri B. Chklovskii, Thomas Schikorski, and Charles F. Stevens. Wiring optimization in cortical circuits. *Neuron*, 34(3):341–347, 2002. doi: 10.1016/S0896-6273(02)00679-7.
- [17] Jeff Clune, Kenneth O. Stanley, Robert T. Pennock, and Charles Ofria. On the performance of indirect encoding across the continuum of regularity. *IEEE Transactions on Evolutionary Computation*, 15(3):346–367, 2011. doi: 10.1109/TEVC.2010.2104157.
- [18] Matthew Cook. Universality in elementary cellular automata. *Complex Systems*, 15(1):1–40, 2004.
- [19] Scott E. Fahlman and Christian Lebiere. The cascade-correlation learning architecture. In *Advances in Neural Information Processing Systems (NIPS 1990)*, volume 2, pages 524–532, 1990.
- [20] Bernd Fritzke. A growing neural gas network learns topologies. In *Advances in Neural Information Processing Systems (NIPS 1995)*, volume 7, pages 625–632, 1995.
- [21] Carlos Gershenson. Guiding the self-organization of random boolean networks. *Theory in Biosciences*, 131(3):181–191, 2012. doi: 10.1007/s12064-011-0144-x.
- [22] Marek Grochowski and Włodzisław Duch. Learning highly non-separable boolean functions using constructive feedforward neural network. In *Proceedings of the 17th International Conference on Artificial Neural Networks (ICANN 2007)*, pages 219–228, 2007. doi: 10.1007/978-3-540-74690-4_23.
- [23] Frédéric Gruau. *Neural Network Synthesis Using Cellular Encoding and the Genetic Algorithm*. PhD thesis, Université Claude Bernard-Lyon I, 1994.
- [24] Nijat Hajizada, Heiko Wersing, and Yaochu Jin. A developmental approach to structural self-organization in reservoir computing. *IEEE Transactions on Cognitive and Developmental Systems*, 2024. Forthcoming.
- [25] Hananel Hazan and others. Neural networks grown and self-organized by noise. *PLOS Computational Biology*, 2024.
- [26] Quincy Hershey and Randy C. Paffenroth. Balancing sparse RNNs with hyperparameterization benefiting meta-learning. *arXiv preprint arXiv:2509.15057*, 2025.
- [27] Russell Impagliazzo, Ramamohan Paturi, and Michael E. Saks. Size-depth tradeoffs for threshold circuits. *SIAM Journal on Computing*, 26(3):693–707, 1997. doi: 10.1137/S0097539794277421.
- [28] Eiji Iyoda, Hajime Nobuhara, and Kaoru Hirota. A solution for the n -bit parity problem using a single neuron with a periodic activation function. In *Proceedings of the IEEE International Conference on Systems, Man and Cybernetics*, 2003.
- [29] Vineet R. Khare, Xin Yao, Bernhard Sendhoff, Yaochu Jin, and Heiko Wersing. Co-evolutionary modular neural networks for automatic problem decomposition. In *Proceedings of the 2005 IEEE Congress on Evolutionary Computation*, volume 3, pages 2691–2698, 2005. doi: 10.1109/CEC.2005.1555051.

- [30] Victor M. Landassuri-Moreno. *Evolution of Modular Neural Networks*. PhD thesis, University of Birmingham, 2012.
- [31] Craig Larman. *Applying UML and patterns: An introduction to object-oriented analysis and design and iterative development*. 2004.
- [32] Yann LeCun, Yoshua Bengio, and Geoffrey Hinton. Deep learning. *Nature*, 521(7553): 436–444, 2015. doi: 10.1038/nature14539.
- [33] Hanxiao Liu, Karen Simonyan, and Yiming Yang. DARTS: Differentiable architecture search. In *International Conference on Learning Representations (ICLR)*, 2019.
- [34] Shiwei Liu, Decebal Constantin Mocanu, Amarsagar Reddy Ramapuram Matavalam, Yulong Pei, and Mykola Pechenizkiy. Sparse evolutionary deep learning with over one million artificial neurons on commodity hardware. *arXiv preprint arXiv:1901.09181*, 2019.
- [35] Dimitar Lyubchev and Angel Marchev. Revisiting and improving the NEAT algorithm. In *2025 IEEE International Conference on Systems, Man, and Cybernetics (SMC)*, 2025.
- [36] Marvin Minsky and Seymour Papert. *Perceptrons: An Introduction to Computational Geometry*. MIT Press, Cambridge, MA, 1969.
- [37] Decebal Constantin Mocanu, Elena Mocanu, Peter Stone, Phuong H. Nguyen, Madeleine Gibescu, and Antonio Liotta. Scalable training of artificial neural networks with adaptive sparse connectivity inspired by network science. *Nature Communications*, 9(1):2383, 2018. doi: 10.1038/s41467-018-04316-3.
- [38] Alexander Mordvintsev, Ettore Randazzo, Eyvind Niklasson, and Michael Levin. Growing neural cellular automata. *Distill*, 2020. doi: 10.23915/distill.00023.
- [39] Kenichi Morita, Yasuyuki Tojima, Katsunobu Imai, and Tsuyoshi Ogiro. Universal computing in reversible and number-conserving two-dimensional cellular spaces. In Andrew Adamatzky, editor, *Collision-Based Computing*, pages 161–199. Springer, London, 2002. doi: 10.1007/978-1-4471-0129-1_6.
- [40] Humphrey Munn and Marcus Gallagher. Towards understanding the link between modularity and performance in neural networks for reinforcement learning. *arXiv preprint arXiv:2205.06451*, 2024.
- [41] Emre O. Neftci, Hesham Mostafa, and Friedemann Zenke. Surrogate gradient learning in spiking neural networks: Bringing the power of gradient-based optimization to spiking neural networks. *IEEE Signal Processing Magazine*, 36(6):51–63, 2019. doi: 10.1109/MSP.2019.2931595.
- [42] Ferda Nur Özçelik and Mehmet Önder Efe. Evolutionary neural architecture search: A survey. *Turkish Journal of Electrical Engineering and Computer Sciences*, 2026.
- [43] David L. Parnas. On the criteria to be used in decomposing systems into modules. *Communications of the ACM*, 15(12):1053–1058, 1972. doi: 10.1145/361598.361623.
- [44] Jorge Pérez, Pablo Barceló, and Javier Marinkovic. Attention is turing complete. *Journal of Machine Learning Research*, 22(75):1–35, 2021.
- [45] Hieu Pham, Melody Y. Guan, Barret Zoph, Quoc V. Le, and Jeff Dean. Efficient neural architecture search via parameter sharing. *Proceedings of the 35th International Conference on Machine Learning (ICML)*, pages 4095–4104, 2018.

- [46] Erwan Plantec, Joachim W. Pedersen, Milton L. Montero, Eleni Nisioti, and Sebastian Risi. Evolving self-assembling neural networks: From spontaneous activity to experience-dependent learning. *arXiv preprint arXiv:2406.09787*, 2024.
- [47] S.-S. Poil and others. Network structure influences self-organized criticality in neural networks with dynamical synapses. *Frontiers in Physics*, 2025.
- [48] Esteban Real, Alok Aggarwal, Yanping Huang, and Quoc V. Le. Regularized evolution for image classifier architecture search. *Proceedings of the AAAI Conference on Artificial Intelligence*, 33:4780–4789, 2019.
- [49] Pengzhen Ren, Yun Xiao, Xiaojun Chang, Po-Yao Huang, Zhihui Li, Xiaojiang Chen, and Xin Wang. A comprehensive survey of neural architecture search: Challenges and solutions. *National Science Review*, 11(8):nwae282, 2024. doi: 10.1093/nsr/nwae282.
- [50] Sebastian Risi and Kenneth O. Stanley. An enhanced hypercube-based encoding for evolving the placement, density, and connectivity of neurons. *Artificial Life*, 18(4):331–363, 2012. doi: 10.1162/ARTL_a_00071.
- [51] Nicolás Rodríguez and others. Neuromorphic hierarchical modular reservoirs. *Nature Communications*, 2026. doi: 10.1038/s41467-026-xxxxx. Forthcoming.
- [52] Frank Rosenblatt. The perceptron: A probabilistic model for information storage and organization in the brain. *Psychological Review*, 65(6):386–408, 1958. doi: 10.1037/h0042519.
- [53] David E. Rumelhart, Geoffrey E. Hinton, and Ronald J. Williams. Learning representations by back-propagating errors. *Nature*, 323(6088):533–536, 1986. doi: 10.1038/323533a0.
- [54] Zubair Shah and Noaman Khan. Pruning as evolution: Emergent sparsity through selection dynamics in neural networks. *arXiv preprint arXiv:2601.10765*, 2026.
- [55] Dhadkan Shrestha and Damian Valles. Reinforced NEAT algorithms for autonomous rover navigation in multi-room dynamic scenario. *Fire*, 8(2):41, 2025. doi: 10.3390/fire8020041.
- [56] Hava T. Siegelmann and Eduardo D. Sontag. On the computational power of neural nets. *Journal of Computer and System Sciences*, 50(1):132–150, 1995. doi: 10.1006/jcss.1995.1013.
- [57] Kenneth O. Stanley and Risto Miikkulainen. Efficient reinforcement learning through evolving neural network topologies. In *Proceedings of the Genetic and Evolutionary Computation Conference (GECCO-2002)*, pages 9–16, San Francisco, CA, 2002. Morgan Kaufmann.
- [58] Kenneth O. Stanley and Risto Miikkulainen. Evolving neural networks through augmenting topologies. *Evolutionary Computation*, 10(2):99–127, 2002. doi: 10.1162/106365602320169811.
- [59] Kenneth O. Stanley, David B. D’Ambrosio, and Jason Gauci. A hypercube-based encoding for evolving large-scale neural networks. *Artificial Life*, 15(2):185–212, 2009. doi: 10.1162/artl.2009.15.2.15202.
- [60] John Stogin, Ankur Mali, and C. Lee Giles. A provably stable neural network turing machine with finite precision and time. *Information and Computation*, 658:120034, 2024. doi: 10.1016/j.ic.2024.105168.

- [61] Suguru Tamaki and others. Average-case lower bounds and satisfiability algorithms for small threshold circuits. In *Proceedings of the 31st Conference on Computational Complexity (CCC 2016)*, 2016.
- [62] Zoltán Toroczkai and Kevin E. Bassler. Network dynamics: Jamming is limited in scale-free systems. *Nature*, 428:716, 2004. doi: 10.1038/428716a.
- [63] Lishuang Wang, Mengfei Zhao, Enyu Liu, and Ran Cheng. Tensorized neuroevolution of augmenting topologies for GPU acceleration. In *Proceedings of the Genetic and Evolutionary Computation Conference (GECCO '24)*, pages 1156–1164, 2024. doi: 10.1145/3638529.3654152.
- [64] David H. Wolpert and William G. Macready. No free lunch theorems for optimization. *IEEE Transactions on Evolutionary Computation*, 1(1):67–82, 1997. doi: 10.1109/4235.585893.
- [65] Barret Zoph and Quoc V. Le. Neural architecture search with reinforcement learning. *arXiv preprint arXiv:1611.01578*, 2017. ICLR 2017.